register file, the ALU, and adders.

deasserted The signal being logically low or false.

deasserted signal A signal that is (logically) false, or 0.

decoder A logic block that has an n-bit input and $2n$ outputs, where only one output is asserted for each input combination.

defect A microscopic flaw in a wafer or in patterning steps that can result in the failure of the die containing that defect.

delayed branch A type of branch where the instruction immediately following the branch is always executed, independent of whether the branch condition is true or false.

die The individual rectangular sections that are cut from a wafer, more informally known as **chips**.

direct-mapped cache A cache structure in which each memory location is mapped to exactly one location in the cache.

dividend A number being divided.

divisor A number that the dividend is divided by.

don't-care term An element of a logical function in which the output does not depend on the values of all the inputs. Don't-care terms may be specified in different ways.

double precision A floating-point value represented in a 64-bit doubleword.

doubleword Another natural unit of access in a computer, usually a group of 64 bits; corresponds to the size of a register in the RISC-V architecture.

dynamic branch prediction Prediction of branches at runtime using runtime information.

dynamic multiple issue An approach to implementing a multiple-issue processor where many decisions are made during execution by the processor.

dynamic pipeline scheduling Hardware support for reordering the order of instruction execution so as to avoid stalls.

www.youseficlass.ir

dynamic random access memory (DRAM) Memory built as an integrated circuit; it provides random access to any location. Access times are 50 nanoseconds and cost per gigabyte in 2012 was $5 to $10.

dynamically linked libraries (DLLs) Library routines that are linked to a program during execution.

edge-triggered clocking A clocking scheme in which all state changes occur on a clock edge.

embedded computer A computer inside another device used for running one predetermined application or collection of software.

EOR A logical bit-by-bit operation with two operands that calculates the exclusive OR of the two operands. That is, it calculates a 1 only if the values are different in the two operands.

error detection code A code that enables the detection of an error in data, but not the precise location and, hence, correction of the error.

exception Also called an **interrupt**. An unscheduled event that disrupts program execution; used to detect overflow.

exception enable Also called interrupt enable. A signal or action that controls whether the process responds to an exception or not; necessary for preventing the occurrence of exceptions during intervals before the processor has safely saved the state needed to restart.

executable file A functional program in the format of an object file that contains no unresolved references. It can contain symbol tables and debugging information. A "stripped executable" does not contain that information. Relocation information may be included for the loader.

exponent In the numerical representation system of floating-point arithmetic, the value that is placed in the exponent field.

external label Also called **global label**. A label referring to an object that can be referenced from files other than the one in

www.youseficlass.ir

which it is defined.

false sharing When two unrelated shared variables are located in the same cache block and the full block is exchanged between processors even though the processors are accessing different variables.

field programmable devices (FPD) An integrated circuit containing combinational logic, and possibly memory devices, that are configurable by the end user.

field programmable gate array (FPGA) A configurable integrated circuit containing both combinational logic blocks and flip-flops.

fine-grained multithreading A version of hardware multithreading that implies switching between threads after every instruction.

finite-state machine A sequential logic function consisting of a set of inputs and outputs, a next-state function that maps the current state and the inputs to a new state, and an output function that maps the current state and possibly the inputs to a set of asserted outputs.

flash memory A nonvolatile semiconductor memory. It is cheaper and slower than DRAM but more expensive per bit and faster than magnetic disks. Access times are about 5 to 50 microseconds and cost per gigabyte in 2012 was $0.75 to $1.00.

flip-flop A memory element for which the output is equal to the value of the stored state inside the element and for which the internal state is changed only on a clock edge.

floating point Computer arithmetic that represents numbers in which the binary point is not fixed.

flush To discard instructions in a pipeline, usually due to an unexpected event.

formal parameter A variable that is the argument to a procedure or macro; replaced by that argument once the macro is expanded.

forward reference A label that is used before it is defined.

www.yousefclass.ir

forwarding Also called **bypassing**. A method of resolving a data hazard by retrieving the missing data element from internal buffers rather than waiting for it to arrive from programmer-visible registers or memory.

fraction The value, generally between 0 and 1, placed in the fraction field.

frame pointer A value denoting the location of the saved registers and local variables for a given procedure.

fully associative cache A cache structure in which a block can be placed in any location in the cache.

fully connected network A network that connects processor-memory nodes by supplying a dedicated communication link between every node.

fused multiply add A floating-point instruction that performs both a multiply and an add, but rounds only once after the add.

gate A device that implements basic logic functions, such as AND or OR.

global miss rate The fraction of references that miss in all levels of a multilevel cache.

global pointer The register that is reserved to point to the static area.

guard The first of two extra bits kept on the right during intermediate calculations of floating-point numbers; used to improve rounding accuracy.

handler Name of a software routine invoked to "handle" an exception or interrupt.

hardware description language A programming language for describing hardware, used for generating simulations of a hardware design and also as input to synthesis tools that can generate actual hardware.

hardware multithreading Increasing utilization of a processor by switching to another thread when one thread is stalled.

www.yousefclass.ir

hardware synthesis tools Computer-aided design software that can generate a gate-level design based on behavioral descriptions of a digital system.

hexadecimal Numbers in base 16.

high-level programming language A portable language such as C, C++, Java, or Visual Basic that is composed of words and algebraic notation that can be translated by a compiler into assembly language.

hit rate The fraction of memory accesses found in a level of the memory hierarchy.

hit time The time required to access a level of the memory hierarchy, including the time needed to determine whether the access is a hit or a miss.

hold time The minimum time during which the input must be valid after the clock edge.

implementation Hardware that obeys the architecture abstraction.

imprecise interrupt Also called **imprecise exception**. Interrupts or exceptions in pipelined computers that are not associated with the exact instruction that was the cause of the interrupt or exception.

in-order commit A commit in which the results of pipelined execution are written to the programmer visible state in the same order that instructions are fetched.

input device A mechanism through which the computer is fed information, such as a microphone.

instruction A command that computer hardware understands and obeys.

instruction count The number of instructions executed by the program.

instruction format A form of representation of an instruction composed of fields of binary numbers.

instruction latency The inherent execution time for an instruction.

www.youseficlass.ir

instruction-level parallelism The parallelism among instructions.

instruction mix A measure of the dynamic frequency of instructions across one or many programs.

instruction set architecture Also called **architecture**. An abstract interface between the hardware and the lowest-level software that encompasses all the information necessary to write a machine language program that will run correctly, including instructions, registers, memory access, I/O, and so on.

integrated circuit Also called a **chip**. A device combining dozens to millions of transistors.

interrupt An exception that comes from outside of the processor. (Some architectures use the term *interrupt* for all exceptions.)

interrupt handler A piece of code that is run as a result of an exception or an interrupt.

issue packet The set of instructions that issues together in one clock cycle; the packet may be determined statically by the compiler or dynamically by the processor.

issue slots The positions from which instructions could issue in a given clock cycle; by analogy, these correspond to positions at the starting blocks for a sprint.

Java bytecode Instruction from an instruction set designed to interpret Java programs.

Just In Time compiler (JIT) The name commonly given to a compiler that operates at runtime, translating the interpreted code segments into the native code of the computer.

latch A memory element in which the output is equal to the value of the stored state inside the element and the state is changed whenever the appropriate inputs change and the clock is asserted.

latency (pipeline) The number of stages in a pipeline or the number of stages between two instructions during execution.

least recently used (LRU) A replacement scheme in which the block replaced is the one that has been unused for the longest

www.youseficlass.ir

time.

least significant bit The rightmost bit in a RISC-V doubleword.

level-sensitive clocking A timing methodology in which state changes occur at either high or low clock levels but are not instantaneous, as such changes are in edge-triggered designs.

linker Also called **link editor**. A systems program that combines independently assembled machine language programs and resolves all undefined labels into an executable file.

liquid crystal display A display technology using a thin layer of liquid polymers that can be used to transmit or block light according to whether a charge is applied.

load-use data hazard A specific form of data hazard in which the data being loaded by a load instruction have not yet become available when they are needed by another instruction.

loader A systems program that places an object program in main memory so that it is ready to execute.

local area network (LAN) A network designed to carry data within a geographically confined area, typically within a single building.

local label A label referring to an object that can be used only within the file in which it is defined.

local miss rate The fraction of references to one level of a cache that miss; used in multilevel hierarchies.

lock A synchronization device that allows access to data to only one processor at a time.

lookup tables (LUTs) In a field programmable device, the name given to the cells because they consist of a small amount of logic and RAM.

loop unrolling A technique to get more performance from loops that access arrays, in which multiple copies of the loop body are made and instructions from different iterations are scheduled together.

www.youseficlass.ir

machine language Binary representation used for communication within a computer system.

macro A pattern-matching and replacement facility that provides a simple mechanism to name a frequently used sequence of instructions.

magnetic disk Also called **hard disk**. A form of nonvolatile secondary memory composed of rotating platters coated with a magnetic recording material. Because they are rotating mechanical devices, access times are about 5 to 20 milliseconds and cost per gigabyte in 2012 was $0.05 to $0.10.

main memory Also called **primary memory**. Memory used to hold programs while they are running; typically consists of DRAM in today's computers.

memory The storage area in which programs are kept when they are running, and that contains the data needed by the running programs.

memory hierarchy A structure that uses multiple levels of memories; as the distance from the processor increases, the size of the memories and the access time both increase.

message passing Communicating between multiple processors by explicitly sending and receiving information.

metastability A situation that occurs if a signal is sampled when it is not stable for the required setup and hold times, possibly causing the sampled value to fall into the indeterminate region between a high and low value.

microarchitecture The organization of the processor, including the major functional units, their interconnection, and control.

million instructions per second (MIPS) A measurement of program execution speed based on the number of millions of instructions. MIPS is computed as the instruction count divided by the product of the execution time and 10^6.

MIMD or Multiple Instruction streams, Multiple Data streams. A multiprocessor.

www.yousefclass.ir

minterms Also called **product terms**. A set of logic inputs joined by conjunction (AND operations); the product terms form the first logic stage of the programmable logic array (PLA).

miss penalty The time required to fetch a block into a level of the memory hierarchy from the lower level, including the time to access the block, transmit it from one level to the other, insert it in the level that experienced the miss, and then pass the block to the requestor.

miss rate The fraction of memory accesses not found in a level of the memory hierarchy.

most significant bit The leftmost bit in a RISC-V doubleword.

multicore microprocessor A microprocessor containing multiple processors ("cores") in a single integrated circuit. Virtually all microprocessors today in desktops and servers are multicore.

multilevel cache A memory hierarchy with multiple levels of caches, rather than just a cache and main memory.

multiple issue A scheme whereby multiple instructions are launched in one clock cycle.

multiprocessor A computer system with at least two processors. This computer is in contrast to a uniprocessor, which has one, and is increasingly hard to find today.

multistage network A network that supplies a small switch at each node.

NAND gate An inverted AND gate.

network bandwidth Informally, the peak transfer rate of a network; can refer to the speed of a single link or the collective transfer rate of all links in the network.

next-state function A combinational function that, given the inputs and the current state, determines the next state of a finite-state machine.

nonblocking assignment An assignment that continues after evaluating the right-hand side, assigning the left-hand side the value only after all right-hand sides are evaluated.

www.youseficlass.ir

nonblocking cache A cache that allows the processor to make references to the cache while the cache is handling an earlier miss.

nonuniform memory access (NUMA) A type of single address space multiprocessor in which some memory accesses are much faster than others depending on which processor asks for which word.

nonvolatile memory A form of memory that retains data even in the absence of a power source and that is used to store programs between runs. A DVD disk is nonvolatile.

nop An instruction that does no operation to change state.

NOR A logical bit-by-bit operation with two operands that calculates the NOT of the OR of the two operands. That is, it calculates a 1 only if there is a 0 in *both* operands.

NOR gate An inverted OR gate.

normalized A number in floating-point notation that has no leading 0s.

NOT A logical bit-by-bit operation with one operand that inverts the bits; that is, it replaces every 1 with a 0, and every 0 with a 1.

object oriented language A programming language that is oriented around objects rather than actions, or data versus logic.

one's complement A notation that represents the most negative value by $10\ldots000_{two}$ and the most positive value by $01\ldots11_{two}$, leaving an equal number of negatives and positives but ending up with two zeros, one positive ($00\ldots00_{two}$) and one negative ($11\ldots11_{two}$). The term is also used to mean the inversion of every bit in a pattern: 0 to 1 and 1 to 0.

opcode The field that denotes the operation and format of an instruction.

OpenMP An API for shared memory multiprocessing in C, C++, or Fortran that runs on UNIX and Microsoft platforms. It includes

www.youseficlass.ir

compiler directives, a library, and runtime directives.

OR A logical bit-by-bit operation with two operands that calculates a 1 if there is a 1 in *either* operand.

out-of-order execution A situation in pipelined execution when an instruction blocked from executing does not cause the following instructions to wait.

output device A mechanism that conveys the result of a computation, such as a display, to a user or to another computer.

page fault An event that occurs when an accessed page is not present in main memory.

page table The table containing the virtual to physical address translations in a virtual memory system. The table, which is stored in memory, is typically indexed by the virtual page number; each entry in the table contains the physical page number for that virtual page if the page is currently in memory.

parallel processing program A single program that runs on multiple processors simultaneously.

PC-relative addressing An addressing regime in which the address is the sum of the program counter (PC) and a constant in the instruction.

personal computer (PC) A computer designed for use by an individual, usually incorporating a graphics display, a keyboard, and a mouse.

personal mobile devices (PMDs) Small wireless devices to connect to the Internet; they rely on batteries for power, and software is installed by downloading apps. Conventional examples are smart phones and tablets.

physical address An address in main memory.

physically addressed cache A cache that is addressed by a physical address.

pipeline stall Also called **bubble**. A stall initiated in order to resolve a hazard.

www.youseficlass.ir

pipelining An implementation technique in which multiple instructions are overlapped in execution, much like an assembly line.

pixel The smallest individual picture element. Screens are composed of hundreds of thousands to millions of pixels, organized in a matrix.

pop Remove element from stack.

precise interrupt Also called **precise exception**. An interrupt or exception that is always associated with the correct instruction in pipelined computers.

prefetching A technique in which data blocks needed in the future are brought into the cache early by the use of special instructions that specify the address of the block.

procedure A stored subroutine that performs a specific task based on the parameters with which it is provided.

procedure call frame A block of memory that is used to hold values passed to a procedure as arguments, to save registers that a procedure may modify but that the procedure's caller does not want changed, and to provide space for variables local to a procedure.

procedure frame Also called **activation record**. The segment of the stack containing a procedure's saved registers and local variables.

process Includes one or more threads, the address space, and the operating system state. Hence, a process switch usually invokes the operating system, but not a thread switch.

program counter (PC) The register containing the address of the instruction in the program being executed.

programmable array logic (PAL) Contains a programmable and-plane followed by a fixed or-plane.

programmable logic array (PLA) A structured-logic element composed of a set of inputs and corresponding input complements and two stages of logic, the first generating

www.youseficlass.ir

product terms of the inputs and input complements, and the second generating sum terms of the product terms. Hence, PLAs implement logic functions as a sum of products.

programmable logic device (PLD) An integrated circuit containing combinational logic whose function is configured by the end user.

programmable ROM (PROM) A form of read-only memory that can be programmed when a designer knows its contents.

propagation time The time required for an input to a flip-flop to propagate to the outputs of the flip-flop.

protection A set of mechanisms for ensuring that multiple processes sharing the processor, memory, or I/O devices cannot interfere, intentionally or unintentionally, with one another by reading or writing each other's data. These mechanisms also isolate the operating system from a user process.

pseudoinstruction A common variation of assembly language instructions often treated as if it were an instruction in its own right.

Pthreads A UNIX API for creating and manipulating threads. It is structured as a library.

push Add element to stack.

quotient The primary result of a division; a number that when multiplied by the divisor and added to the remainder produces the dividend.

read-only memory (ROM) A memory whose contents are designated at creation time, after which the contents can only be read. ROM is used as structured logic to implement a set of logic functions by using the terms in the logic functions as address inputs and the outputs as bits in each word of the memory.

receive message routine A routine used by a processor in machines with private memories to accept a message from another processor.

www.youseficlass.ir

recursive procedures Procedures that call themselves either directly or indirectly through a chain of calls.

reduction A function that processes a data structure and returns a single value.

reference bit Also called **use bit** or **access bit**. A field that is set whenever a page is accessed and that is used to implement LRU or other replacement schemes.

reg In Verilog, a register.

register file A state element that consists of a set of registers that can be read and written by supplying a register number to be accessed.

register renaming The renaming of registers by the compiler or hardware to remove antidependences.

register use convention Also called **procedure call convention**. A software protocol governing the use of registers by procedures.

relocation information The segment of a UNIX object file that identifies instructions and data words that depend on absolute addresses.

remainder The secondary result of a division; a number that when added to the product of the quotient and the divisor produces the dividend.

reorder buffer The buffer that holds results in a dynamically scheduled processor until it is safe to store the results to memory or a register.

reservation station A buffer within a functional unit that holds the operands and the operation.

response time Also called **execution time**. The total time required for the computer to complete a task, including disk accesses, memory accesses, I/O activities, operating system overhead, CPU execution time, and so on.

restartable instruction An instruction that can resume execution after an exception is resolved without the exceptions affecting the result of the instruction.

www.yousefclass.ir

return address A link to the calling site that allows a procedure to return to the proper address; in RISC-V, it is usually stored in register `x1`.

rotational latency Also called **rotational delay**. The time required for the desired sector of a disk to rotate under the read/write head; usually assumed to be half the rotation time.

round Method to make the intermediate floating-point result fit the floating-point format; the goal is typically to find the nearest number that can be represented in the format. It is also the name of the second of two extra bits kept on the right during intermediate floating-point calculations, which improves rounding accuracy.

scientific notation A notation that renders numbers with a single digit to the left of the decimal point.

secondary memory Nonvolatile memory used to store programs and data between runs; typically consists of flash memory in PMDs and magnetic disks in servers.

sector One of the segments that make up a track on a magnetic disk; a sector is the smallest amount of information that is read or written on a disk.

seek The process of positioning a read/write head over the proper track on a disk.

segmentation A variable-size address mapping scheme in which an address consists of two parts: a segment number, which is mapped to a physical address, and a segment offset.

selector value Also called **control value**. The control signal that is used to select one of the input values of a multiplexor as the output of the multiplexor.

semiconductor A substance that does not conduct electricity well.

send message routine A routine used by a processor in machines with private memories to pass a message to another processor.

sensitivity list The list of signals that specifies when an always block should be re-evaluated.

www.yousefclass.ir

separate compilation Splitting a program across many files, each of which can be compiled without knowledge of what is in the other files.

sequential logic A group of logic elements that contain memory and hence whose value depends on the inputs as well as the current contents of the memory.

server A computer used for running larger programs for multiple users, often simultaneously, and typically accessed only via a network.

set-associative cache A cache that has a fixed number of locations (at least two) where each block can be placed.

setup time The minimum time that the input to a memory device must be valid before the clock edge.

shared memory multiprocessor (SMP) A parallel processor with a single physical address space.

sign-extend Increases the size of a data item by replicating the high-order sign bit of the original data item in the high-order bits of the larger, destination data item.

silicon A natural element that is a semiconductor.

silicon crystal ingot A rod composed of a silicon crystal that is between 8 and 12 inches in diameter and about 12 to 24 inches long.

SIMD or Single Instruction stream, Multiple Data streams. The same instruction is applied to many data streams, as in a vector processor.

simple programmable logic device (SPLD) Programmable logic device, usually containing either a single PAL or PLA.

simultaneous multithreading (SMT) A version of multithreading that lowers the cost of multithreading by utilizing the resources needed for multiple issue, dynamically scheduled microarchitecture.

single precision A floating-point value represented in a 32-bit word.

www.youseficlass.ir

single-cycle implementation Also called **single clock cycle implementation**. An implementation in which an instruction is executed in one clock cycle. While easy to understand, it is too slow to be practical.

SISD or Single Instruction stream, Single Data stream. A uniprocessor.

Software as a Service (SaaS) delivers software and data as a service over the Internet, usually via a thin program such as a browser that runs on local client devices, instead of binary code that must be installed, and runs wholly on that device. Examples include web search and social networking.

source language The high-level language in which a program is originally written.

spatial locality The locality principle stating that if a data location is referenced, data locations with nearby addresses will tend to be referenced soon.

speculation An approach whereby the compiler or processor guesses the outcome of an instruction to remove it as a dependence in executing other instructions.

split cache A scheme in which a level of the memory hierarchy is composed of two independent caches that operate in parallel with each other, with one handling instructions and one handling data.

SPMD Single Program, Multiple Data streams. The conventional MIMD programming model, where a single program runs across all processors.

stack A data structure for spilling registers organized as a last-in-first-out queue.

stack pointer A value denoting the most recently allocated address in a stack that shows where registers should be spilled or where old register values can be found. In RISC-V, it is register `x2`, also known as sp.

stack segment The portion of memory used by a program to hold procedure call frames.

www.youseficlass.ir

state element A memory element, such as a register or a memory.

static data The portion of memory that contains data whose size is known to the compiler and whose lifetime is the program's entire execution.

static multiple issue An approach to implementing a multiple-issue processor where many decisions are made by the compiler before execution.

static random access memory (SRAM) A memory where data are stored statically (as in flip-flops) rather than dynamically (as in DRAM). SRAMs are faster than DRAMs, but less dense and more expensive per bit.

sticky bit A bit used in rounding in addition to guard and round that is set whenever there are nonzero bits to the right of the round bit.

stored-program concept The idea that instructions and data of many types can be stored in memory as numbers and thus be easy to change, leading to the stored program computer.

strong scaling Speed-up achieved on a multiprocessor without increasing the size of the problem.

structural hazard When a planned instruction cannot execute in the proper clock cycle because the hardware does not support the combination of instructions that are set to execute.

structural specification Describes how a digital system is organized in terms of a hierarchical connection of elements.

sum of products A form of logical representation that employs a logical sum (OR) of products (terms joined using the AND operator).

supercomputer A class of computers with the highest performance and cost; they are configured as servers and typically cost tens to hundreds of millions of dollars.

superscalar An advanced pipelining technique that enables the processor to execute more than one instruction per clock cycle by selecting them during execution.

www.youseficlass.ir

supervisor mode Also called **kernel mode**. A mode indicating that a running process is an operating system process.

swap space The space on the disk reserved for the full virtual memory space of a process.

symbol table A table that matches names of labels to the addresses of the memory words that instructions occupy.

synchronization The process of coordinating the behavior of two or more processes, which may be running on different processors.

synchronizer failure A situation in which a flip-flop enters a metastable state and where some logic blocks reading the output of the flip-flop see a 0 while others see a 1.

synchronous system A memory system that employs clocks and where data signals are read only when the clock indicates that the signal values are stable.

system call A special instruction that transfers control from user mode to a dedicated location in supervisor code space, invoking the exception mechanism in the process.

system CPU time The CPU time spent in the operating system performing tasks on behalf of the program.

systems software Software that provides services that are commonly useful, including operating systems, compilers, loaders, and assemblers.

tag A field in a table used for a memory hierarchy that contains the address information required to identify whether the associated block in the hierarchy corresponds to a requested word.

task-level parallelism or process-level parallelism Utilizing multiple processors by running independent programs simultaneously.

temporal locality The principle stating that if a data location is referenced, then it will tend to be referenced again soon.

terabyte (TB) Originally 1,099,511,627,776 (240) bytes, although communications and secondary storage systems developers

www.youseficlass.ir

started using the term to mean 1,000,000,000,000 (10^{12}) bytes. To reduce confusion, we now use the term **tebibyte** (TiB) for 240 bytes, defining terabyte (TB) to mean 10^{12} bytes. (Figure 1.1 shows the full range of decimal and binary values and names.)

text segment The segment of a UNIX object file that contains the machine language code for routines in the source file.

thread A thread includes the program counter, the register state, and the stack. It is a lightweight process; whereas threads commonly share a single address space, processes don't.

three Cs model A cache model in which all cache misses are classified into one of three categories: compulsory misses, capacity misses, and conflict misses.

throughput Also called **bandwidth**. Another measure of performance, it is the number of tasks completed per unit time.

tournament branch predictor A branch predictor with multiple predictions for each branch and a selection mechanism that chooses which predictor to enable for a given branch.

track One of thousands of concentric circles that makes up the surface of a magnetic disk.

transistor An on/off switch controlled by an electric signal.

translation-lookaside buffer (TLB) A cache that keeps track of recently used address mappings to try to avoid an access to the page table.

truth table From logic, a representation of a logical operation by listing all the values of the inputs and then in each case showing what the resulting outputs should be.

underflow (floating-point) A situation in which a negative exponent becomes too large to fit in the exponent field.

uniform memory access (UMA) A multiprocessor in which latency to any word in main memory is about the same no matter which processor requests the access.

units in the last place (ulp) The number of bits in error in the least significant bits of the significand between the actual number

www.youseficlass.ir

and the number that can be represented.

unmapped A portion of the address space that cannot have page faults.

unresolved reference A reference that requires more information from an outside source to be complete.

use bit Also called **reference bit** or **access bit**. A field that is set whenever a page is accessed and that is used to implement LRU or other replacement schemes.

use latency Number of clock cycles between a load instruction and an instruction that can use the result of the load without stalling the pipeline.

user CPU time The CPU time spent in a program itself.

valid bit A field in the tables of a memory hierarchy that indicates that the associated block in the hierarchy contains valid data.

vector lane One or more vector functional units and a portion of the vector register file. Inspired by lanes on highways that increase traffic speed, multiple lanes execute vector operations simultaneously.

vectored interrupt An interrupt for which the address to which control is transferred is determined by the cause of the exception.

Verilog One of the two most common hardware description languages.

very-large-scale integrated (VLSI) circuit A device containing hundreds of thousands to millions of transistors.

very long instruction word (VLIW) A style of instruction set architecture that launches many operations that are defined to be independent in a single wide instruction, typically with many separate opcode fields.

VHDL One of the two most common hardware description languages.

virtual address An address that corresponds to a location in

www.youseficlass.ir

virtual space and is translated by address mapping to a physical address when memory is accessed.

virtual machine A virtual computer that appears to have nondelayed branches and loads and a richer instruction set than the actual hardware.

virtual memory A technique that uses main memory as a "cache" for secondary storage.

virtually addressed cache A cache that is accessed with a virtual address rather than a physical address.

volatile memory Storage, such as DRAM, that retains data only if it is receiving power.

wafer A slice from a silicon ingot no more than 0.1 inches thick, used to create chips.

weak scaling Speed-up achieved on a multiprocessor while expanding the size of the problem proportionally to the increase in the number of processors.

wide area network (WAN) A network extended over hundreds of kilometers that can span a continent.

wire In Verilog, specifies a combinational signal.

word The natural unit of access in a computer, usually a group of 32 bits.

workload A set of programs run on a computer that is either the actual collection of applications run by a user or constructed from real programs to approximate such a mix. A typical workload specifies both the programs and the relative frequencies.

write buffer A queue that holds data while the data are waiting to be written to memory.

write-back A scheme that handles writes by updating values only to the block in the cache, then writing the modified block to the lower level of the hierarchy when the block is replaced.

write-through A scheme in which writes always update both the

www.yousefclass.ir

cache and the next lower level of the memory hierarchy, ensuring that data are always consistent between the two.

yield The percentage of good dies from the total number of dies on the wafer.

www.yousefclass.ir

Further Reading

Chapter 1

1. Barroso L, Hölzle U. The case for energy-proportional computing. *IEEE Computer*. 2007;December.

A plea to change the nature of computer components so that they use much less power when lightly utilized.

2. Bell CG. *Computer Pioneers and Pioneer Computers* ACM and the Computer Museum, videotapes 1996.

Two videotapes on the history of computing, produced by Gordon and Gwen Bell, including the following machines and their inventors: Harvard Mark-I, ENIAC, EDSAC, IAS machine, and many others.

3. Burks, A.W., H.H. Goldstine, and J. von Neumann [1946]. "Preliminary discussion of the logical design of an electronic computing instrument," *Report to the U.S. Army Ordnance Department*, p. 1; also appears in *Papers of John von Neumann*, W. Aspray and A. Burks (Eds.), MIT Press, Cambridge, MA, and Tomash Publishers, Los Angeles, 1987, 97–146.

A classic paper explaining computer hardware and software before the first stored-program computer was built. We quote extensively from it in Chapter 3. It simultaneously explained computers to the world and was a source of controversy because the first draft did not give credit to Eckert and Mauchly.

4. Campbell-Kelly M, Aspray W. *Computer: A History of the Information Machine* New York: Basic Books; 1996.

Two historians chronicle the dramatic story. The New York Times *calls it well written and authoritative*.

5. Ceruzzi PF. *A History of Modern Computing* Cambridge, MA: MIT Press; 1998.

www.youseficlass.ir

Contains a good description of the later history of computing: the integrated circuit and its impact, personal computers, UNIX, and the Internet.

6. Curnow HJ, Wichmann BA. A synthetic benchmark. *The Computer J.* 1976;19(1):80.

Describes the first major synthetic benchmark, Whetstone, and how it was created.

7. Flemming PJ, Wallace JJ. How not to lie with statistics: The correct way to summarize benchmark results. *Commun ACM*. 1986;29(3 (March)):218–221.

Describes some of the underlying principles in using different means to summarize performance results.

8. Goldstine HH. *The Computer: From Pascal to von Neumann* Princeton, NJ: Princeton University Press; 1972.

A personal view of computing by one of the pioneers who worked with von Neumann.

9. Hayes B. Computing in a parallel universe. *American Scientist*. 2007;Vol. 95(November–December):476–480.

An overview of the parallel computing challenge written for the layman.

10. Hennessy JL, Patterson DA. *Chapter 1 of Computer Architecture: A Quantitative Approach* fifth edition Waltham, MA: Morgan Kaufmann Publishers; 2012.

Section 1.5 goes into more detail on power, Section 1.6 contains much more detail on the cost of integrated circuits and explains the reasons for the difference between price and cost, and Section 1.8 gives more details on evaluating performance.

11. Lampson, B.W. [1986]. "Personal distributed computing; The Alto and Ethernet software." In ACM Conference on the History of Personal Workstations (January).
12. Thacker, C.R. [1986]. "Personal distributed computing: The Alto and Ethernet hardware," In ACM Conference on the History of Personal Workstations (January).

These two papers describe the software and hardware of the landmark Alto.

13. Metropolis N, Howlett J, Rota G-C, eds. *A History of Computing in the Twentieth Century*. New York: Academic Press; 1980.

A collection of essays that describe the people, software, computers, and laboratories involved in the first experimental and commercial computers.

www.yousefclass.ir

Most of the authors were personally involved in the projects. An excellent bibliography of early reports concludes this interesting book.

14. Public Broadcasting System [1992]. *The Machine That Changed the World*, videotapes.

These five 1-hour programs include rare footage and interviews with pioneers of the computer industry.

15. Slater R. *Portraits in Silicon* Cambridge, MA: MIT Press; 1987.

Short biographies of 31 computer pioneers.

16. Stern N. Who invented the first electronic digital computer? *Annals of the History of Computing*. 1980;2(4 (October)):375–376.

A historian s perspective on Atanasoff versus Eckert and Mauchly.

17. Wilkes MV. *Memoirs of a Computer Pioneer* Cambridge, MA: MIT Press; 1985.

A personal view of computing by one of the pioneers.

Chapter 2

1. Bayko, J. [1996]. "Great microprocessors of the past and present," search for it on the *http://www.cpushack.com/CPU/cpu.html*.

A personal view of the history of both representative and unusual microprocessors, from the Intel 4004 to the Patriot Scientific ShBoom!

2. Kane G, Heinrich J. *MIPS RISC Architecture* Englewood Cliffs, NJ: Prentice Hall; 1992.

This book describes the MIPS architecture in greater detail than Appendix A.

3. Levy H, Eckhouse R. *Computer Programming and Architecture: The VAX* Boston: Digital Press; 1989.

This book concentrates on the VAX, but also includes descriptions of the Intel 8086, IBM 360, and CDC 6600.

4. Morse S, Ravenal B, Mazor S, Pohlman W. Intel microprocessors—8080 to 8086. *Computer*. 1980;13.

The architecture history of the Intel from the 4004 to the 8086, according to the people who participated in the designs.

5. Wakerly J. *Microcomputer Architecture and Programming* New York: Wiley; 1989.

The Motorola 6800 is the main focus of the book, but it covers the Intel 8086, Motorola 6809, TI 9900, and Zilog Z8000.

www.youseficlass.ir

6. Waterman, A. Y. Lee, D. Patterson, and K. Asanović [2016]. The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.1.

The canonical reference manual for the RISC-V instruction set architecture, this technical report discusses the rationale behind the myriad tradeoffs in the ISA's design. Download from http://riscv.org/specifications/.

Chapter 3

If you are interested in learning more about floating point, two publications by David Goldberg [1991, 2002] are good starting points; they abound with pointers to further reading. Several of the stories told in this section come from Kahan [1972, 1983]. The latest word on the state of the art in computer arithmetic is often found in the *Proceedings* of the latest IEEE-sponsored Symposium on Computer Arithmetic, held every 2 years; the 16th was held in 2003.

1. Burks, A.W., H.H. Goldstine, and J. von Neumann [1946]. "Preliminary discussion of the logical design of an electronic computing instrument," *Report to the U.S. Army Ordnance Dept*., p. 1; also in *Papers of John von Neumann*, W. Aspray and A. Burks (Eds.), MIT Press, Cambridge, MA, and Tomash Publishers, Los Angeles, 1987, 97–146.

This classic paper includes arguments against floating-point hardware.

2. Goldberg, D. [2002]. "Computer arithmetic". Appendix J of *Computer Architecture: A Quantitative Approach*, fifth edition, J. L. Hennessy and D. A. Patterson, Morgan Kaufmann Publishers, Waltham, MA.

A more advanced introduction to integer and floating-point arithmetic, with emphasis on hardware. It covers Sections 3.4–3.6 of this book in just 10 pages, leaving another 45 pages for advanced topics.

3. Goldberg D. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*. 1991;23(1):5–48.

Another good introduction to floating-point arithmetic by the same author, this time with emphasis on software.

4. Kahan W. *A survey of error-analysis*. *Info Processing 71* (Proc. IFIP Congress 71 in Ljubljana). Vol. 2 Amsterdam: North-Holland Publishing; 1972;1214–1239.

www.youseficlass.ir

This survey is a source of stories on the importance of accurate arithmetic.

5. Kahan W. Mathematics written in sand. *Proc Amer Stat Assoc Joint Summer Meetings of 1983, Statistical Computing Section* 1983;12–26.

The title refers to silicon and is another source of stories illustrating the importance of accurate arithmetic.

6. Kahan W. *"On the advantage of the 8087's stack," unpublished course notes, Computer Science Division* Berkeley: University of California; 1990.

What the 8087 floating-point architecture could have been.

7. Kahan, W. [1997]. Available at http://www.cims.nyu.edu/~dbindel/class/cs279/87stack.pdf.

A collection of memos related to floating point, including "Beastly numbers" (another less famous Pentium bug), "Notes on the IEEE floating point arithmetic" (including comments on how some features are atrophying), and "The baleful effects of computing benchmarks" (on the unhealthy preoccupation on speed versus correctness, accuracy, ease of use, flexibility, …).

8. Koren I. *Computer Arithmetic Algorithms* second edition Natick, MA: A. K. Peters; 2002.

A textbook aimed at seniors and first-year graduate students that explains fundamental principles of basic arithmetic, as well as complex operations such as logarithmic and trigonometric functions.

9. Wilkes MV. *Memoirs of a Computer Pioneer* Cambridge, MA: MIT Press; 1985.

This computer pioneer's recollections include the derivation of the standard hardware for multiply and divide developed by von Neumann.

Chapter 4

1. Bhandarkar D, Clark DW. *Performance from architecture: Comparing a RISC and a CISC with similar hardware organizations. Proc Fourth Conf on Architectural Support for Programming Languages and Operating Systems* Palo Alto, CA: IEEE/ACM (April); 1991;310–319.

A quantitative comparison of RISC and CISC written by scholars who argued for CISCs as well as built them; they conclude that MIPS is between 2 and 4 times faster than a VAX built with similar technology,

www.yousefclass.ir

with a mean of 2.7.

2. Fisher JA, Rau BR. *Journal of Supercomputing (January)* Kluwer 1993.

This entire issue is devoted to the topic of exploiting ILP. It contains papers on both the architecture and software and is a wonderful source for further references.

3. Hennessy JL, Patterson DA. *Computer Architecture: A Quantitative Approach* fifth edition Waltham, MA: Morgan Kaufmann; 2012.

Chapter 3 and Appendix C go into considerably more detail about pipelined processors (almost 200 pages), including superscalar processors and VLIW processors. Appendix G describes Itanium.

4. Jouppi NP, Wall DW. *Available instruction-level parallelism for superscalar and superpipelined processors*. *Proc Third Conf on Architectural Support for Programming Languages and Operating Systems* Boston: IEEE/ACM (April); 1989;272–282.

A comparison of deeply pipelined (also called superpipelined) and superscalar systems.

5. Kogge PM. *The Architecture of Pipelined Computers* New York: McGraw-Hill; 1981.

A formal text on pipelined control, with emphasis on underlying principles.

6. Russell RM. The CRAY-1 computer system. *Commun ACM*. 1978;21(1 (January)):63–72.

A short summary of a classic computer that uses vectors of operations to remove pipeline stalls.

7. Smith A, Lee J. Branch prediction strategies and branch target buffer design. *Computer*. 1984;17(1 (January)):6–22.

An early survey on branch prediction.

8. Smith JE, Plezkun AR. Implementing precise interrupts in pipelined processors. *IEEE Trans on Computers*. 1988;37(5 (May)):562–573.

Covers the difficulties in interrupting pipelined computers.

9. Thornton JE. *Design of a Computer: The Control Data 6600* Glenview, IL: Scott, Foresman; 1970.

A classic book describing a classic computer, considered the first supercomputer.

Chapter 5

www.youseficlass.ir

1. Cantin JF, Hill MD. Cache performance for selected SPEC CPU2000 benchmarks. *SIGARCH Computer Architecture News*. 2001;29(4 (September)):13–18.

A reference paper of cache miss rates for many cache sizes for the SPEC2000 benchmarks.

2. Conti C, Gibson DH, Pitowsky SH. Structural aspects of the System/360 Model 85, part I: General organization. *IBM Systems J.* 1968;7(1):2–14.

A classic paper that describes the first commercial computer to use a cache and its resulting performance.

3. Hennessy J, Patterson D. *Chapter 2 and Appendix B in Computer Architecture: A Quantitative Approach* fifth edition Waltham, MA: Morgan Kaufmann Publishers; 2012.

For more in-depth coverage of a variety of topics including protection, cache performance of out-of-order processors, virtually addressed caches, multilevel caches, compiler optimizations, additional latency tolerance mechanisms, and cache coherency.

4. Kilburn T, Edwards DBG, Lanigan MJ, Sumner FH. *One-level storage system. IRE Transactions on Electronic Computers* 1962; EC-11 (April), 223–35. Also appears in D. P. Siewiorek, C G Bell, and A. Newell [1982], *Computer Structures: Principles and Examples*, McGraw-Hill, New York, 135–148.

This classic paper is the first proposal for virtual memory.

5. LaMarca A, Ladner RE. The influence of caches on the performance of heaps. *ACM J of Experimental Algorithmics*. 1996;Vol. 1.

This paper shows the difference between complexity analysis of an algorithm, instruction count performance, and memory hierarchy for four sorting algorithms.

6. McCalpin, J.D. [1995]. "STREAM: Sustainable Memory Bandwidth in High Performance Computers," https://www.cs.virginia.edu/stream/.

A widely used microbenchmark that measures the performance of the memory system behind the caches.

7. Przybylski SA. *Cache and Memory Hierarchy Design: A Performance-Directed Approach* San Francisco: Morgan Kaufmann Publishers; 1990.

A thorough exploration of multilevel memory hierarchies and their performance.

www.youseficlass.ir

8. Ritchie D. The evolution of the UNIX time-sharing system. *AT& T Bell Laboratories Technical Journal*. 1984;1984:1577–1593.

The history of UNIX from one of its inventors.

9. Ritchie DM, Thompson K. The UNIX time-sharing system. *Bell System Technical Journal* 1978;(August):1991–2019.

A paper describing the most elegant operating system ever invented.

10. Silberschatz A, Galvin P, Grange G. *Operating System Concepts* sixth edition Reading, MA: Addison-Wesley; 2003.

An operating systems textbook with a thorough discussion of virtual memory processes and process management, and protection issues.

11. Smith AJ. Cache memories. *Computing Surveys*. 1982;14(3 (September)):473–530.

The classic survey paper on caches. This paper defined the terminology for the field and has served as a reference for many computer designers.

12. Smith DK, Alexander RC. *Fumbling the Future: How Xerox Invented, Then Ignored, the First Personal Computer* New York: Morrow; 1988.

A popular book that explains the role of Xerox PARC in laying the foundation for today's computing, but which Xerox did not substantially benefit from.

13. Tanenbaum A. *Modern Operating Systems* second edition Upper Saddle River: Prentice Hall, NJ; 2001.

An operating system textbook with a good discussion of virtual memory.

14. Waterman, A. Y. Lee, D. Patterson, and K. Asanović [2016]. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.9.1.

The RISC-V Privileged Architecture manual discusses in more detail the layered privilege mode design and the memory address-translation and protection schemes described in Chapter 5.

15. Wilkes M. Slave memories and dynamic storage allocation. *IEEE Trans Electronic Computers EC*. 1965;14(2 (April)):270–271.

The first classic paper on caches.

Chapter 6

1. Almasi GS, Gottlieb A. *Highly Parallel Computing* Redwood

www.youseficlass.ir

City, CA: Benjamin/Cummings; 1989.
A textbook covering parallel computers.

2. Amdahl, G.M. [1967]. "Validity of the single processor approach to achieving large scale computing capabilities," *Proc. AFIPS Spring Joint Computer Conf.*, Atlantic City, NJ (April), 483–85.

Written in response to the claims of the Illiac IV, this three-page article describes Amdahl's law and gives the classic reply to arguments for abandoning the current form of computing.

3. Andrews GR. *Concurrent Programming: Principles and Practice* Redwood City, CA: Benjamin/Cummings; 1991.

A text that gives the principles of parallel programming.

4. Archibald J, Baer J-L. Cache coherence protocols: Evaluation using a multiprocessor simulation model. *ACM Trans on Computer Systems*. 1986;4(4 (November)):273–298.

Classic survey paper of shared-bus cache coherence protocols.

5. Arpaci-Dusseau, A., R. Arpaci-Dusseau, D. Culler, J. Hellerstein, and D. Patterson [1997]. "High-performance sorting on networks of workstations," *Proc. ACM SIG MOD/PODS Conference on Management of Data*, Tucson, AZ (May), 12–15.

How a world record sort was performed on a cluster, including architecture critique of the workstation and network interface. By April 1, 1997, they pushed the record to 8.6 GB in 1 minute and 2.2 seconds to sort 100 MB.

6. Asanovic K, Bodik R, Catanzaro BC, et al. *The landscape of parallel computing research: A view from Berkeley*. *Tech Rep UCB/EECS-2006-183* Berkeley: EECS Department, University of California; 2006; (December 18).

Nicknamed the "Berkeley View," this report lays out the landscape of the multicore challenge.

7. Bailey, D.H., E. Barszcz, J.T. Barton, D.S. Browning, R.L. Carter, L. Dagum, R.A. Fatoohi, P.O. Frederickson, T.A. Lasinski, R.S. Schreiber, H.D. Simon, V. Venkatakrishnan, and S.K. Weeratunga. [1991]. "The NAS parallel benchmarks—summary and preliminary results," *Proceedings of the 1991 ACM/IEEE Conference on Supercomputing* (August).

Describes the NAS parallel benchmarks.

www.youseficlass.ir

8. Bell CG. Multis: A new class of multiprocessor computers. *Science*. 1985;228(April 26):462–467.

Distinguishes shared address and nonshared address multiprocessors based on micro-processors.

9. Bienia, C., S. Kumar, J.P. Singh, and K. Li [2008]. "The PARSEC benchmark suite: characterization and architectural implications," Princeton University Technical Report TR-81 1-008 (January).

Describes the PARSEC parallel benchmarks. Also see http://parsec.cs.princeton.edu/.

10. Cooper, B.F., A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears [2010]. Benchmarking cloud serving systems with YCSB, In: *Proceedings of the 1st ACM Symposium on Cloud Computing*, Indianapolis, Indiana, USA. http://dx.doi.org/10.1145/1807128.1807152 (June).

Presents the "Yahoo! Cloud Serving Benchmark" (YCSB) framework, with the goal of facilitating performance comparisons of the new generation of cloud data serving systems.

11. Culler DE, Singh JP, Gupta A. *Parallel Computer Architecture* San Francisco: Morgan Kaufmann; 1998.

A textbook on parallel computers.

12. Dongarra JJ, Bunch JR, Moler GB, Stewart GW. *LINPACK Users' Guide* Society for Industrial Mathematics 1979.

The original document describing Linpack, which became a widely used parallel benchmark.

13. Falk H. Reaching for the gigaflop. *IEEE Spectrum*. 1976;13(10 (October)):65–70.

Chronicles the sad story of the Illiac IV: four times the cost and less than one-tenth the performance of original goals.

14. Flynn MJ. Very high-speed computing systems. *Proc IEEE*. 1966;54(12 (December)):1901–1909.

Classic article showing SISD/SIMD/MISD/MIMD classifications.

15. Hennessy J, Patterson D. *Chapters 5 and Appendices F and I. Computer Architecture: A Quantitative Approach* fifth edition Waltham, MA: Morgan Kaufmann Publishers; 2012.

A more in-depth coverage of a variety of multiprocessor and cluster topics, including programs and measurements.

16. Henning JL. SPEC CPU suite growth: an historical perspective. *Computer Architecture News*. 2007;Vol. 35

www.youseficlass.ir

(March).

Gives the history of SPEC, including the use of SPECrate to measure performance on independent jobs, which is being used as a parallel benchmark.

17. Hord RM. *The Illiac-IV, the First Supercomputer* Rockville, MD: Computer Science Press; 1982.

A historical accounting of the Illiac IV project.

18. Hwang K. *Advanced Computer Architecture with Parallel Programming* New York: McGraw-Hill; 1993.

Another textbook covering parallel computers.

19. Kozyrakis C, Patterson D. Scalable vector processors for embedded systems. *IEEE Micro*. 2003;23(6 (November–December)):36–45.

Examination of a vector architecture for the MIPS instruction set in media and signal processing.

20. Laprie, J.-C. [1985]. "Dependable computing and fault tolerance: Concepts and terminology," *15th Annual Int'l Symposium on Fault-Tolerant Computing FTCS 15*, Digest of Papers, Ann Arbor, MI (June 19–21) 2–11.

The paper that introduced standard definitions of dependability, reliability, and availability.

21. Menabrea LF. Sketch of the analytical engine invented by Charles Babbage. *Bibliothèque Universelle de Genève* 1842; (October).

Certainly the earliest reference on multiprocessors, this mathematician made this comment while translating papers on Babbage's mechanical computer.

22. Patterson, D., G. Gibson, and R. Katz [1988]. "A case for redundant arrays of inexpensive disks (RAID)," *SIGMOD Conference*, 109–16.

23. Pfister GF. *In Search of Clusters: The Coming Battle in Lowly Parallel Computing* second edition Upper Saddle River, NJ: Prentice Hall; 1998.

An entertaining book that advocates clusters and is critical of NUMA multiprocessors.

24. Regnier G, Makineni S, Illikkal R, et al. TCP onloading for data center servers. *IEEE Computer*. 2004;37(11):48–58.

Describes the work of researchers at Intel Labs, who have experimented with alternative solutions that improve the server's ability to process

www.youseficlass.ir

TCP/IP packets efficiently and at very high rates.

25. Seitz C. The Cosmic Cube. *Comm ACM*. 1985;28(1 (January)):22–31.

A tutorial article on a parallel processor connected via a hypertree. The Cosmic Cube is the ancestor of the Intel supercomputers.

26. Slotnick DL. The conception and development of parallel processors—a personal memoir. *Annals of the History of Computing*. 1982;4(1 (January)):20–30.

Recollections of the beginnings of parallel processing by the architect of the Illiac IV.

27. Williams S, Waterman A, Patterson D. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*. 2009;52(4 (April)):65–76.

28. Williams, S., J. Carter, L. Oliker, J. Shalf, and K. Yelick [2008]. "Lattice Boltzmann simulation optimization on leading multicore platforms," *International Parallel & Distributed Processing Symposium (IPDPS)*.

Paper containing the results of the four multicores for LBMHD.

29. Williams S, Oliker L, Vuduc R, Shalf J, Yelick K, Demmel J. Optimization of sparse matrix-vector multiplication on emerging multicore platforms. *Supercomputing (SC)* 2007.

Paper containing the results of the four multicores for SPmV.

30. Williams, S. [2008]. *Autotuning Performance of Multicore Computers*, Ph.D. Dissertation, U.C. Berkeley.

Dissertation containing the roofline model.

31. Woo, S.C., M. Ohara, E. Torrie, J.P. Singh, and A. Gupta. "The SPLASH-2 programs: characterization and methodological considerations," *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA '95)*, May, 24–36.

Paper describing the second version of the Stanford parallel benchmarks.

Appendix A

There are a number of good texts on logic design. Here are some you might like to look into.

1. Ashenden P. *Digital Design: An Embedded Systems Approach Using VHDL/Verilog* Waltham, MA: Morgan Kaufmann;

www.youseficlass.ir

2007.
2. Ciletti MD. *Advanced Digital Design with the Verilog HDL* Englewood Cliff s, NJ: Prentice Hall; 2002.
A thorough book on logic design using Verilog.
3. Harris D, Harris S. *Digital Design and Computer Architecture* Waltham, MA: Morgan Kaufmann; 2012.
A unique and modern approach to digital design using VHDL and SystemVerilog.
4. Katz RH. *Modern Logic Design* 2nd ed. Reading, MA: Addison-Wesley; 2004.
A general text on logic design.
5. Wakerly JF. *Digital Design: Principles and Practices* 3rd ed. Englewood Cliff s, NJ: Prentice Hall; 2000.
A general text on logic design.

Appendix B

1. Akeley, K. and T. Jermoluk [1988]. "High-Performance Polygon Rendering," *Proc. SIGGRAPH 1988* (August), 239–46.
2. Akeley, K. [1993]. "RealityEngine Graphics." *Proc. SIGGRAPH 1993* (August), 109–16.
3. Blelloch GB. Prefix Sums and Their Applications. In: Reif John H, ed. *Synthesis of Parallel Algorithms*. San Francisco: Morgan Kaufmann Publishers; 1990.
4. Blythe D. The Direct3D 10 System. *ACM Trans Graphics*. 2006;Vol. 25 (July), 724–734.
5. Buck, I., T. Foley, D. Horn, J. Sugerman, K. Fatahlian, M. Houston, and P. Hanrahan [2004]. "Brook for GPUs: Stream Computing on Graphics Hardware." *Proc. SIGGRAPH 2004*, 777–86, August. http://doi.acm.org/10.1145/1186562.1015800.
6. Elder, G. [2002] "Radeon 9700." Eurographics/SIGGRAPH Workshop on Graphics Hardware, Hot3D Session. www.graphicshardware.org/previous/www_2002/presentat RADEON9700.ppt.
7. Fernando R, Kilgard MJ. *The Cg Tutorial: The Definitive Guide to Programmable Real-Time Graphics* Reading, MA: Addison-Wesley; 2003.

www.yousefclass.ir

8. Fernando R, ed. *GPU Gems: Programming Techniques, Tips, and Tricks for Real-Time Graphics*. Reading, MA: Addison-Wesley; 2004; https://developer.nvidia.com/gpugems/GPUGems/gpugems
9. Foley J, van Dam A, Feiner S, Hughes J. *Computer Graphics: Principles and Practice, second edition in C* Reading, MA: Addison- Wesley; 1995.
10. Hillis WD, Steele GL. Data parallel algorithms. *Commun ACM*. 1986;29(12 (Dec.)):1170–1183 http://doi.acm.org/10.1145/7902.7903.
11. IEEE 754R Working Group [2006]. *DRAFT Standard for Floating-Point Arithmetic P754*. www.validlab.com/754R/drafts/archive/2006-10-04.pdf.
12. Industrial Light and Magic [2003]. *OpenEXR*, www.openexr.com.
13. Intel Corporation [2007]. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. November. Order Number: 248966-016. Also: http://www.intel.com/content/dam/www/public/us/en/docu ia-32-architectures-optimization-manual.pdf.
14. Kessenich, J. [2006]. *The OpenGL Shading Language, Language Version 1.20, Sept. 2006*. www.opengl.org/documentation/specs/.
15. Kirk, D. and D. Voorhies [1990]. "The Rendering Architecture of the DN10000VS." *Proc. SIGGRAPH* 1990 (August), 299–307.
16. Lindholm E., M.J. Kilgard, and H. Moreton [2001]. "A User-Programmable Vertex Engine." *Proc. SIGGRAPH* 2001 (August), 149–58.
17. Lindholm E, Nickolls J, Oberman S, Montrym J. NVIDIA Tesla: A Unified Graphics and Computing Architecture. *IEEE Micro*. 2008;Vol. 28 (March–April), 39–55.
18. Microsoft Corporation. *Microsoft DirectX Specification*, https://msdn.microsoft.com/en-us/library/windows/apps/hh452744.aspx.
19. Microsoft Corporation. *Microsoft DirectX 9 Programmable Graphics Pipeline* Redmond, WA: Microsoft Press; 2003.
20. Montrym, J., D. Baum, D. Dignam, and C. Migdal [1997]. "InfiniteReality: A Real-Time Graphics System." *Proc.*

www.youseficlass.ir

SIGGRAPH 1997 (August), 293–301.
21. Montrym J, Moreton H. The GeForce 6800. *IEEE Micro*. 2005;Vol. 25 (March–April), 41–51.
22. Moore GE. Cramming more components onto integrated circuits. *Electronics*. 1965;Vol. 38 (April 19).
23. Nguyen H, ed. *GPU Gems 3*. Reading, MA: Addison-Wesley; 2008.
24. Nickolls J, Buck I, Garland M, Skadron K. Scalable Parallel Programming with CUDA. *ACM Queue*. 2008;Vol. 6(no. 2 (March–April)):40–53.
25. NVIDIA [2007]. CUDA Zone. http://www.nvidia.com/object/cuda_home_new.html.
26. NVIDIA [2007]. *CUDA Programming Guide 1.1*. https://developer.nvidia.com/nvidia-gpu-programming-guide.
27. NVIDIA [2007]. *PTX: Parallel Thread Execution ISA version 1.1*. www.nvidia.com/object/io_1195170102263.html.
28. Nyland, L., M. Harris, and J. Prins [2007]. "Fast N-Body Simulation with CUDA". In *GPU Gems 3*, H. Nguyen (Ed.), Addison-Wesley, Reading, MA.
29. Oberman, S.F. and M.Y. Siu [2005]. "A High-Performance Area-Efficient Multifunction Interpolator," *Proc. Seventeenth IEEE Symp. Computer Arithmetic*, 272–79.
30. Pharr M, ed. *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation*. Reading, MA: Addison-Wesley; 2005.
31. Satish, N., M. Harris, and M. Garland [2008]. "Designing Efficient Sorting Algorithms for Manycore GPUs," NVIDIA Technical Report NVR-2008-001.
32. Segal, M. and K. Akeley [2006]. *The OpenGL Graphics System: A Specification, Version 2.1, Dec. 1, 2006*. www.opengl.org/documentation/specs/.
33. Sengupta, S., M. Harris, Y. Zhang, and J.D. Owens [2007]. "Scan Primitives for GPU Computing." In *Proc. of Graphics Hardware 2007* (August), 97–106.
34. Volkov, V. and J. Demmel [2008]. "LU, QR and Cholesky Factorizations using Vector Capabilities of GPUs," Technical Report No. UCB/EECS-2008-49, 1–11. http://www.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-

www.yousefclass.ir

2008-49.pdf.
35. Williams, S., L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel [2007]. "Optimization of sparse matrix-vector multiplication on emerging multicore platforms," In *Proc. Supercomputing 2007*, November.

Appendix D

1. Bhandarkar DP. *Alpha Architecture and Implementations* Newton, MA: Digital Press; 1995.
2. Darcy, J.D., and D. Gay [1996]. "FLECKmarks: Measuring floating point performance using a compliant arithmetic benchmark," CS 252 class project, U.C Berkeley (see *www.sonic.net/~jddarcy/Research/fleckmrk.pdf*)).
3. Digital Semiconductor. *Alpha Architecture Handbook, Version 3* Maynard, MA: Digital Press; 1996; Order number EC-QD2KB-TE (October).
4. Furber SB. *ARM System Architecture* Harlow, England: Addison-Wesley; 1996; (See *http://www.pearsonhighered.com/pearsonhigheredus/educator/pro isbn=9780201675191&forced_logout=forced_logged_out#sthash.C*
5. Hewlett-Packard [1994]. *PA-RISC 2.0 Architecture Reference Manual*, 3rd ed.
6. Hitachi [1997]. *SuperH RISC Engine SH7700 Series Programming Manual*. (See *http://am.renesas.com/products/mpumcu/superh/sh7700/Documer*
7. IBM. *The PowerPC Architecture* San Francisco: Morgan Kaufmann; 1994.
8. Kane G. *PA-RISC 2.0 Architecture* Upper Saddle River, NJ: Prentice Hall PTR; 1996.
9. Kane G, Heinrich J. *MIPS RISC Architecture* Englewood Cliffs, NJ: Prentice Hall; 1992.
10. Kissell, K.D. [1997]. *MIPS16: High-Density for the Embedded Market*.
11. Magenheimer DJ, Peters L, Pettis KW, Zuras D. Integer multiplication and division on the HP precision architecture. *IEEE Trans on Computers*. 1988;37(8):980–990.
12. MIPS [1997]. *MIPS16 Application Specific Extension Product Description*.

www.youseficlass.ir

13. Mitsubishi [1996]. *Mitsubishi 32-Bit Single Chip Microcomputer M32R Family Software Manual* (September).
14. Muchnick SS. Optimizing compilers for SPARC. *Sun Technology*. 1988;1(3 (Summer)):64–77.
15. Seal, D. *Arm Architecture Reference Manual*, 2nd ed, Morgan Kaufmann, 2000.
16. Silicon Graphics [1996]. MIPS V Instruction Set.
17. Sites RL, Witek R, eds. *Alpha Architecture Reference Manual*. 2nd ed. Newton, MA: Digital Press; 1995.
18. Sloss, A. N., D. Symes, and C. Wright, *ARM System Developer's Guide*, San Francisco: Elsevier Morgan Kaufmann, 2004.
19. Sun Microsystems [1989]. *The SPARC Architectural Manual*, Version 8, Part No. 800-1399-09, August 25.
20. Sweetman, D. *See MIPS Run*, 2nd ed, Morgan Kaufmann, 2006.
21. Taylor, G., P. Hilfinger, J. Larus, D. Patterson, and B. Zorn [1986]. "Evaluation of the SPUR LISP architecture," *Proc. 13th Symposium on Computer Architecture* (June), Tokyo.
22. Ungar, D., R. Blau, P. Foley, D. Samples, and D. Patterson [1984]. "Architecture of SOAR: Smalltalk on a RISC," *Proc. 11th Symposium on Computer Architecture* (June), Ann Arbor, MI, 188–97.
23. Weaver DL, Germond T. *The SPARC Architectural Manual, Version 9* Englewood Cliffs, NJ: Prentice Hall; 1994.
24. Weiss S, Smith JE. *Power and PowerPC* San Francisco: Morgan Kaufmann; 1994.

www.youseficlass.ir

Index

Note: Online information is listed by print page number and a period followed by "e" with online page number (54.e1). Page references preceded by a single letter with hyphen refer to appendices. Page references followed by "*f*," "*t*," and "*b*" refer to figures, tables, and boxes, respectively.

0-9, and symbols

1-bit ALU, A-26–A-29
See also Arithmetic logic unit (ALU)
adder, A-27*f*
CarryOut, A-28
for most significant bit, A-33*f*
illustrated, A-29*f*
logical unit for AND/OR, A-27*f*
performing AND, OR, and addition, A-31, A-33*f*

64-bit ALU, A-29–A-31
See also Arithmetic logic unit (ALU)
from 63 copies of 1-bit ALU, A-34*f*
with 64 1-bit ALUs, A-30*f*
defining in Verilog, A-36–A-37
illustrated, A-35*f*
ripple carry adder, A-29

www.youseficlass.ir

7090/7094 hardware, 227.e6

A

Absolute references, 127

Abstractions

hardware/software interface, 22

principle, 22

to simplify design, 11

Accumulator architectures, 162.e1–162.e2

Acronyms, 9

Active matrix, 18

add (add), 64*f*

addi (add immediate), 64*f*, 72, 84

Addition, 174–177

See also Arithmetic

binary, 174*b*–175*b*

floating-point, 198–201, 206

operands, 175, 175

significands, 197*b*–198*b*

speed, 177*b*

Address interleaving, 372–373

Address select logic, C-24, C-25*f*

Address space, 420, 423*b*

extending, 469*b*

flat, 469

ID (ASID), 438

inadequate, 473.e5–473.e6

shared, 509–510

www.youseficlass.ir

single physical, 509, 509–510
virtual, 438
Address translation
for ARM cortex-A53, 460*f*
defined, 420–421
fast, 430–432
for Intel core i7, 460*f*
TLB for, 430–432
Address-control lines, C-26*f*
Addresses
base, 69
byte, 70
defined, 68
memory, 78*b*
virtual, 420–421, 440, 441*b*
Addressing
base, 118*f*
in branches, 115–117
displacement, 118
immediate, 118*f*
PC-relative, 115–116, 118*f*
register, 118*f*
RISC-V modes, 117–118
x86 modes, 151
Addressing modes
desktop architectures, D-5–D-6
Advanced Vector Extensions (AVX), 218, 219
AGP, B-9–B-10

www.yousefclass.ir

Algol-60, 162.e6
Aliasing, 436
Alignment restriction, 70
All-pairs N-body algorithm, B-65
Alpha architecture
  bit count instructions, D-29
  floating-point instructions, D-28–D-29
  instructions, D-27–D-29
  no divide, D-28
  PAL code, D-28
  unaligned load-store, D-28
  VAX floating-point formats, D-29
ALU control, 251–253
  See also Arithmetic logic unit (ALU)
  bits, 252–253, 252*f*
  logic, C-6–C-7
  mapping to gates, C-4–C-7
  truth tables, C-5*f*, C-5*f*
ALU control block, 255
  defined, C-4–C-6
  generating ALU control bits, C-6*f*
ALUOp, 252, C-6*b*–C-7*b*
  bits, 252, 253
  control signal, 255
Amazon Web Services (AWS), 417*b*
AMD Opteron X4 (Barcelona), 535, 536*f*
AMD64, 148, 148, 217, 162.e5
Amdahl's law, 393, 495–496

www.yousefclass.ir

corollary, 49
defined, 49
fallacy, 548
and (and), 64*f*
AND gates, A-12–A-13, C-7
AND operation, 90, A-6
andi (and immediate), 64*f*
Annual failure rate (AFR), 410–411
versus MTTF of disks, 410*b*–411*b*
Antidependence, 327
Antifuse, A-77
Apple computer, 54.e6
Apple iPad 2 A1395, 20*f*
logic board of, 20*f*
processor integrated circuit of, 21*f*
Application binary interface (ABI), 22
Application programming interfaces (APIs)
defined, B-4
graphics, B-14
Architectural registers, 337–338
Arithmetic, 172
addition, 174–177
addition and subtraction, 174–177
division, 183–191
fallacies and pitfalls, 222–225
floating-point, 191–216
historical perspective, 227
multiplication, 177–183

www.yousefclass.ir

parallelism and, 216–217
Streaming SIMD Extensions and advanced vector extensions in x86, 217–218
subtraction, 174–177
subword parallelism, 216–217
subword parallelism and matrix multiply, 218–222
Arithmetic instructions
See also Instructions
desktop RISC, D-11*f*, D-11*f*
embedded RISC, D-13*f*
logical, 243–244
operands, 67–74
Arithmetic intensity, 533–534
Arithmetic logic unit (ALU)
See also ALU control, Control units
1-bit, A-26–A-29
64-bit, A-29–A-31
before forwarding, 299*f*
branch datapath, 246–247
hardware, 176
memory-reference instruction use, 237
for register values, 244
R-format operations, 245*f*
signed-immediate input, 302
ARM Cortex-A53, 236, 334–342
address translation for, 460*f*
caches in, 461*f*
data cache miss rates for, 462*f*
memory hierarchies of, 459

www.yousefclass.ir

performance of, 462–464
specification, 335*f*
TLB hardware for, 460*f*
ARPAnet, 54.e9
Arrays, 407*f*
logic elements, A-18–A-20
multiple dimension, 212
pointers *versus*, 141–144
procedures for setting to zero, 141*f*
ASCII
binary numbers *versus*, 109*b*
character representation, 108*f*
defined, 108–109
symbols, 111
Assemblers, 125–127
defined, 14
function, 125–127
microcode, C-30
number acceptance, 126
object file, 126
Assembly language, 15*f*
defined, 14, 125
floating-point, 207*f*
illustrated, 15*f*
programs, 125
RISC-V, 64*f*, 85*b*–86*b*
translating into machine language, 85*b*–86*b*
Asserted signals, 242, A-4

www.youseficlass.ir

Associativity
in caches, 397*b*–398*b*
degree, increasing, 396–398, 444
increasing, 401–402
set, tag size *versus*, 401*b*–402*b*
Atomic compare and swap, 123*b*
Atomic exchange, 122
Atomic fetch-and-increment, 123*b*
Atomic memory operation, B-21
Attribute interpolation, B-43–B-44
auipc's effect, 156
Automobiles, computer application in, 4
Average memory access time (AMAT), 394
calculating, 394*b*

B

Bandwidth, 29–30
bisection, 527
external to DRAM, 390
memory, 390
network, 525–526
Barrier synchronization, B-18
defined, B-20
for thread communication, B-34
Base addressing, 69, 118
Base registers, 69
Basic block, 95*b*
Benchmarks, 530–540

www.yousefclass.ir

defined, 46
Linpack, 530, 227.e2–227.e3, 227.e3
multiprocessor, 530–540
NAS parallel, 532
parallel, 531*f*
PARSEC suite, 532
SPEC CPU, 46–48
SPEC power, 48–49
SPECrate, 530
Stream, 540*b*
Biased notation, 81, 195
Binary numbers, 82
ASCII *versus*, 109*b*
conversion to decimal numbers, 77*b*
defined, 74
Bisection bandwidth, 527
Bit maps
defined, 18
goal, 18
storing, 18
Bit-Interleaved Parity (RAID 3), 458.e4
Bits
ALUOp, 252, 253
defined, 14
dirty, 430*b*
guard, 214
patterns, 214*b*–215*b*
reference, 428*b*

www.youseficlass.ir

rounding, 214
sign, 75
state, C-8–C-10
sticky, 214
valid, 376–378
Blocking assignment, A-24
Blocking factor, 406
Block-Interleaved Parity (RAID 4), 458.e4–458.e5
Blocks
combinational, A-4–A-5
defined, 367–368
finding, 444–445
flexible placement, 394–398
least recently used (LRU), 401
locating in cache, 399–401
miss rate and, 383*f*
multiword, mapping addresses to, 382*b*–383*b*
placement locations, 443
placement strategies, 396
replacement selection, 401
replacement strategies, 446
spatial locality exploitation, 383
state, A-4–A-5
valid data, 376–378
Bonding, 28
Boolean algebra, A-6–A-7
Bounds check shortcut, 96
Branch datapath

www.youseficlass.ir

ALU, 246–247
operations, 246–247
Branch if Equal (beq), A-32
Branch if greater than or equal, unsigned (bgeu), 95–96
Branch if less than (blt) instruction, 95–96
Branch if less than, unsigned (bltu), 95–96
Branch instructions
pipeline impact, 308*f*
Branch not taken
assumption, 307–308
defined, 246
Branch prediction
buffers, 310
as control hazard solution, 274
defined, 273–274
dynamic, 274, 310–314
static, 324
Branch predictors
accuracy, 312
correlation, 312–313
information from, 312–313
tournament, 313–314
Branch table, 97–98
Branch taken
cost reduction, 308–309
defined, 246
Branch target
addresses, 246

www.youseficlass.ir

buffers, 312
Branches
See also Conditional branches
addressing in, 115–117
compiler creation, 93–94
decision, moving up, 308–309
delayed, 274, 308–310
ending, 95*b*
execution in ID stage, 309
pipelined, 310*b*
target address, 308–309
Branch-on-zero instruction, 260–261
Bubble Sort, 140
Bubbles, 305
Bus-based coherent multiprocessors, 553.e1
Buses, A-18–A-19
Bytes
addressing, 70
order, 70

C

C language
assignment, compiling into RISC-V, 65*b*
compiling, 144–145, 144.e1–144.e2
compiling assignment with registers, 67*b*–68*b*
compiling while loops in, 94*b*–95*b*
sort algorithms, 141*f*
translation hierarchy, 124*f*

www.yousefclass.ir

translation to RISC-V assembly language, 65
variables, 104*b*
C.mmp, 553.e3–553.e4
C++ language, 162.e7, 144.e26
Cache blocking and matrix multiply, 465–468
Cache coherence, 454–458
coherence, 454
consistency, 454
enforcement schemes, 456
implementation techniques, 459.e10–459.e11
migration, 456
problem, 454, 455*f*, 458*b*
protocol example, 459.e11–459.e15
protocols, 456
replication, 456
snooping protocol, 456–458
snoopy, 459.e16
state diagram, 459.e15*f*
Cache coherency protocol, 459.e11–459.e15
finite-state transition diagram, 459.e14*f*
functioning, 459.e13*f*
mechanism, 459.e13*f*
state diagram, 459.e15*f*
states, 459.e12
write-back cache, 459.e14*f*
Cache controllers, 459
coherent cache implementation techniques, 459.e10–459.e11
implementing, 459.e1

www.yousefclass.ir

snoopy cache coherence, 459.e16
SystemVerilog, 459.e1–459.e4
Cache hits, 460
Cache misses
block replacement on, 445–446
capacity, 447, 448
compulsory, 447
conflict, 447
defined, 384
direct-mapped cache, 396
fully associative cache, 398
handling, 384–385
memory-stall clock cycles, 391
reducing with flexible block placement, 394–398
set-associative cache, 397
steps, 385
in write-through cache, 385
Cache performance, 390–410
calculating, 392*b*–393*b*
hit time and, 393–394
impact on processor performance, 392–393
Cache-aware instructions, 472
Caches, 375–390
See also Blocks
accessing, 378–384
in ARM cortex-A53, 461*f*
associativity in, 397*b*–398*b*
bits in, 382*b*

www.yousefclass.ir

bits needed for, 382
contents illustration, 379*f*
defined, 19–22, 375–376
direct-mapped, 376, 377*f*, 382, 394
empty, 378
FSM for controlling, 449–454
fully associative, 395
GPU, B-38
inconsistent, 385
index, 380
in Intel Core i7, 461*f*
Intrinsity FastMATH example, 387–389
locating blocks in, 399–401
locations, 377*f*
multilevel, 390, 402–405
nonblocking, 460
physically addressed, 436–437
physically indexed, 436*b*–437*b*
physically tagged, 436*b*–437*b*
primary, 402, 409–410
secondary, 402, 409–410
set-associative, 395
simulating, 468*b*
size, 381–383
split, 389*b*
summary, 389–390
tag field, 380
tags, 459.e1*f*, 459.e10–459.e11, 459.e11

www.yousefclass.ir

virtual memory and TLB integration, 435–437
virtually addressed, 436
virtually indexed, 436
virtually tagged, 436
write-back, 386, 387, 446
write-through, 385, 387, 446
writes, 385–387
Callee, 99, 101
Caller, 99
Capabilities, 473.e12
Capacity misses, 447
Carry lookahead, A-37–A-47
4-bit ALUs using, A-43*f*
adder, A-38
fast, with first level of abstraction, A-38–A-40
fast, with "infinite" hardware, A-38
fast, with second level of abstraction, A-40–A-45
plumbing analogy, A-41*f*, A-42*f*
ripple carry speed *versus*, A-45*b*
summary, A-45–A-47
Carry save adders, 183
CDC 6600, 347.e2, 54.e6
Cell phones, 6–7
Central processor unit (CPU)
See also Processors
classic performance equation, 36–40
defined, 19
execution time, 32, 33–34

www.youseficlass.ir

performance, 33–35
system, time, 32
time, 391
time measurements, 33–34
user, time, 32
Cg pixel shader program, B-15
Characters
ASCII representation, 108–109
in Java, 111–113
Chips, 19, 25–26, 26
manufacturing process, 26
Classes
defined, 144.e14
packages, 144.e20
Clock cycles
defined, 33
memory-stall, 391
number of registers and, 67
worst-case delay and, 262
Clock cycles per instruction (CPI), 35–36, 272
one level of caching, 402
two levels of caching, 402
Clock rate
defined, 33
frequency switched as function of, 41
power and, 40
Clocking methodology, 241–243, A-47
edge-triggered, 241, A-47, A-72–A-73

www.youseficlass.ir

level-sensitive, A-73–A-74, A-74–A-75
for predictability, 241
Clocks, A-47–A-49
edge, A-47, A-49*b*
in edge-triggered design, A-72*f*
skew, A-73
specification, A-56*f*
synchronous system, A-47–A-48
Cloud computing, 524–525
defined, 7
Cluster networking, 529–530, 529.e1, 529.e3–529.e5, 529.e6–529.e9
Clusters, 553.e7–553.e8
defined, 492, 522, 553.e7
isolation, 523
organization, 491
scientific computing on, 553.e7
Cm*, 553.e3–553.e4
CMOS (complementary metal oxide semiconductor), 41
Coarse-grained multithreading, 506–507
Cobol, 162.e6
Code generation, 144.e12
Code motion, 144.e6
Cold-start miss, 447
Collision misses, 447
Column major order, 405
Combinational blocks, A-4–A-5
Combinational control units, C-4–C-8
Combinational elements, 240

www.yousefclass.ir

Combinational logic, 241, A-3–A-4, A-9–A-20
arrays, A-18–A-19
decoders, A-9–A-10
defined, A-4–A-5
don’t cares, A-17–A-18
multiplexors, A-10
ROMs, A-14–A-16
two-level, A-11–A-14
Verilog, A-23–A-26
Commercial computer development, 54.e3–54.e9
Commit units
buffer, 329
defined, 329
in update control, 334*b*
Common case fast, 11
Common subexpression elimination, 144.e5
Communication, 23–24
overhead, reducing, 44–45
thread, B-34
Compact code, 162.e3–162.e4
Compare and branch zero, 309
Comparisons
constant operands in, 72–74
signed *versus* unsigned, 95–96
Compilers, 125
branch creation, 94*b*
brief history, 162.e7–162.e8
conservative, 144.e6

www.youseficlass.ir

defined, 14
front end, 144.e2
function, 14, 125
high-level optimizations, 144.e3–144.e4
ILP exploitation, 347.e4–347.e5
Just In Time (JIT), 133
optimization, 141, 162.e8
speculation, 323–324
structure, 144.e1*f*
Compiling
C assignment statements, 65*b*
C language, 94*b*–95*b*, 144–145, 144.e1, 144.e2
floating-point programs, 208*b*–209*b*
if-then-else, 93*b*
in Java, 144.e18–144.e19
procedures, 100*b*–101*b*, 102*b*–103*b*
recursive procedures, 102*b*–103*b*
while loops, 94*b*–95*b*
Compressed sparse row (CSR) matrix, B-55, B-56
Compulsory misses, 447, 448–449
Computer architects, 11–13
abstraction to simplify design, 11
common case fast, 11
dependability via redundancy, 12
hierarchy of memories, 12
Moore's law, 11
parallelism, 12
pipelining, 12

www.youseficlass.ir

prediction, 12
Computers
application classes, traditional, 3
applications, 4
arithmetic for, 172
characteristics, 54.e12*f*
commercial development, 54.e3–54.e9
component organization, 17*f*
components, 17*f*
design measure, 53
desktop, 5
embedded, 5–6
first, 54.e2
in information revolution, 4
instruction representation, 81–89
performance measurement, 54.e1–54.e3
post-PC era, 6–7
servers, 5
Condition codes/flags, 96
Conditional branches
changing program counter with, 312*b*
compiling if-then-else into, 93*b*
defined, 92–93
desktop RISC, D-16*f*
embedded RISC, D-16*f*
implementation, 97*b*
in loops, 117
PA-RISC, D-34–D-36, D-35*f*

www.youseficlass.ir

PC-relative addressing, 115–116
RISC, D-10–D-16
SPARC, D-10–D-12
Conditional move instructions, 313*b*–314*b*
Conflict misses, 447
Constant memory, B-40
Constant operands, 72–74
frequent occurrence, 72
Content Addressable Memory (CAM), 400*b*–401*b*
Context switch, 438*b*
Control
ALU, 251–253
challenge, 315
finalizing, 261
forwarding, 300
FSM, C-8–C-22
implementation, optimizing, C-27
mapping to hardware, C-3–C-4, C-4–C-8, C-8–C-22, C-22–C-28, C-28–C-32, C-32–C-33
memory, C-26*f*
organizing, to reduce logic, C-31–C-32
pipelined, 290–294
Control and status register (CSR) access instructions, 464–465
Control flow graphs, 144.e8, 144.e9
illustrated examples, 144.e8*f*, 144.e9*f*, 144.e11*f*
Control functions
ALU, mapping to gates, C-4–C-7
defining, 256

www.youseficlass.ir

PLA, implementation, C-7, C-20
ROM, encoding, C-19
for single-cycle implementation, 261–262
Control hazards, 271–274, 307–315
branch delay reduction, 308–310
branch not taken assumption, 307–308
branch prediction as solution, 274
delayed decision approach, 274*b*
dynamic branch prediction, 310–314
logic implementation in Verilog, 345.e8
pipeline stalls as solution, 272*f*
pipeline summary, 314–315
solutions, 272*f*
static multiple-issue processors and, 324
Control lines
asserted, 256
in datapath, 255*f*
execution/address calculation, 291
final three stages, 293*f*
instruction decode/register file read, 291
instruction fetch, 291
memory access, 291
setting of, 256
values, 291
write-back, 291
Control signals
ALUOp, 255
defined, 242

www.yousefclass.ir

effect of, 256*f*
multi-bit, 256
pipelined datapaths with, 290–294
truth tables, C-14*f*
Control units, 239–240
See also Arithmetic logic unit (ALU)
address select logic, C-24, C-25*f*
combinational, implementing, C-4–C-8
with explicit counter, C-23*f*
illustrated, 257*f*
logic equations, C-11–C-12
main, designing, 253–256
as microcode, C-28*f*
next-state outputs, C-10, C-12*b*–C-13*b*
output, 251–253, C-10
RISC-V, C-10*f*
Cooperative thread arrays (CTAs), B-30
Coprocessors
defined, 212*b*
Core RISC-V instruction set
abstract view, 238*f*
desktop RISC, D-9*f*
implementation, 236–237
implementation illustration, 239*f*
overview, 237–240
subset, 236
Cores
defined, 43

www.yousefclass.ir

number per chip, 43
Correlation predictor, 312–313
Cosmic Cube, 553.e6–553.e7
CPU, 9
Cray computers, 227.e4, 227.e5
Critical word first, 384
Crossbar networks, 527–528
CTSS (Compatible Time-Sharing System), 473.e13
CUDA programming environment, 515, B-5–B-6
barrier synchronization, B-18, B-34
development, B-17, B-17–B-18
hierarchy of thread groups, B-18
kernels, B-19, B-24
key abstractions, B-18
paradigm, B-19–B-22
parallel plus-scan template, B-61*f*
per-block shared memory, B-58
plus-reduction implementation, B-63*f*
programs, B-6, B-24, B-24
scalable parallel programming with, B-17–B-23
shared memories, B-18
threads, B-36
Cyclic redundancy check, 415*b*–416*b*
Cylinder, 374

D

D flip-flops, A-50–A-51, A-52
D latches, A-50–A-51, A-51

www.yousefclass.ir

Data bits, 413*f*
Data flow analysis, 144.e8
Data hazards, 268–271, 294–307
See also Hazards
forwarding, 268–269, 294–307
load-use, 269–271, 308
stalls and, 303–307
Data parallel problem decomposition, B-17, B-18*f*
Data race, 121
Data selectors, 237–238
Data transfer instructions
See also Instructions
defined, 68, 69
load, 69
offset, 69
store, 70–71
Datacenters, 7
Data-level parallelism, 500
Datapath elements
defined, 243
sharing, 248–249
Datapaths
branch, 246–247
building, 243–251
control signal truth tables, C-14*f*
control unit, 257*f*
defined, 19
design, 243

www.youseficlass.ir

exception handling, 318*f*
for fetching instructions, 245*f*
for hazard resolution via forwarding, 302*f*
for memory instructions, 247
in operation for branch-if-equal instruction, 260–261
in operation for load instruction, 259*f*
in operation for R-type instruction, 258*f*
operation of, 256–261
pipelined, 276–294
for RISC-V architecture, 249
for R-type instructions, 256–259
single, creating, 247–251
single-cycle, 275
static two-issue, 326*f*
Deasserted signals, 242, A-4
DEC PDP-8, 162.e2*f*
Decimal numbers
binary number conversion to, 77*b*
defined, 74
Decision-making instructions, 92–98
Decoders, A-9–A-10
two-level, A-64
Decoding machine language, 118–120
Defect, 26–27
Delayed branches, 274
See also Branches
as control hazard solution, 274
embedded RISCs and, D-23

www.youseficlass.ir

reducing, 308–310
Delayed decision, 274*b*
DeMorgan's theorems, A-11
Denormalized numbers, 216
Dependability via redundancy, 12
Dependable memory hierarchy, 410–416
failure, defining, 410–412
Dependences
between pipeline registers, 238–239
between pipeline registers and ALU inputs, 297–298
bubble insertion and, 305
detection, 297*b*
name, 327
sequence, 295
Design
compromises and, 84
datapath, 243
digital, 345
logic, 240–243
main control unit, 253–256
memory hierarchy, challenges, 449*f*
pipelining instruction sets, 267
Desktop and server RISCs
See also Reduced instruction set computer (RISC) architectures
addressing modes, D-6
architecture summary, D-4*f*, D-4*f*
arithmetic/logical instructions, D-11*f*
conditional branches, D-16

www.yousefclass.ir

constant extension summary, D-9*f*, D-9*f*
control instructions, D-11*f*
conventions equivalent to MIPS core, D-12*f*
data transfer instructions, D-10*f*
features added to, D-45*f*
floating-point instructions, D-12*f*
instruction formats, D-7*f*
multimedia extensions, D-16–D-18
multimedia support, D-18*f*
Desktop computers, defined, 5
Device driver, 529.e4
DGEMM (Double precision General Matrix Multiply), 218–219, 342, 344–345, 405, 530
cache blocked version of, 407*f*
optimized C version of, 220*f*, 342*f*, 466*f*
performance, 344*f*, 408*f*
Dicing, 27
Dies, 26–27
Digital design pipeline, 345
Digital signal-processing (DSP) extensions, D-19
DIMMs (dual inline memory modules), 473.e4
Direct Data IO (DDIO), 529.e6
Direct memory access (DMA), 529.e2*f*, 529.e3
Direct3D, B-13
Direct-mapped caches
See also Caches
address portions, 399*f*
choice of, 400–401

www.youseficlass.ir

defined, 376, 394
illustrated, 377*f*
memory block location, 395*f*
misses, 397*b*–398*b*
single comparator, 399
total number of bits, 382
Dirty bit, 430*b*
Dirty pages, 430*b*
Disk memory, 373–375
Displacement addressing, 118
Distributed Block-Interleaved Parity (RAID 5), 458.e5–458.e6
Divide algorithm, 186*b*
Dividend, 184
Division, 183–191
algorithm, 185*f*
dividend, 184
divisor, 184
Divisor, 184
divu (Divide Unsigned)
See also Arithmetic
faster, 188–189
floating-point, 206–212
hardware, 184–187
hardware, improved version, 187*f*
operands, 184
quotient, 184
remainder, 184
in RISC-V, 189

www.youseficlass.ir

signed, 187–188
SRT, 189
Don't cares, A-17–A-18
example, A-17*b*–A-18*b*
term, 253
Double data rate (DDR), 371–372
Double Data Rate (DDR) SDRAM, 371–372, A-64
Double precision
See also Single precision
defined, 193
FMA, B-45, B-45–B-46
GPU, B-45, B-74*b*
representation, 212–214
Doubleword, 67, 151
Dual inline memory modules (DIMMs), 373
Dynamic branch prediction, 310–314
See also Control hazards
branch prediction buffer, 310
loops and, 312*b*
Dynamic hardware predictors, 274
Dynamic multiple-issue processors, 322, 328–333
See also Multiple issue
pipeline scheduling, 329–333
superscalar, 328
Dynamic pipeline scheduling, 329–333
commit unit, 329
concept, 329
hardware-based speculation, 331–333
primary units, 330*f*

www.youseficlass.ir

reorder buffer, 334*b*
reservation station, 329
Dynamic random access memory (DRAM), 370, 371–373, A-62–A-64
bandwidth external to, 390
cost, 23
defined, 19, A-62
DIMM, 473.e4
Double Date Rate (DDR), 371–372
early board, 473.e4*f*
GPU, B-37–B-38
growth of capacity, 25*f*
history, 473.e1
internal organization of, 372*f*
pass transistor, A-62*b*–A-64*b*
SIMM, 473.e5*f*, 473.e4
single-transistor, A-63*f*
size, 390
speed, 23–24
synchronous (SDRAM), 371–372, A-59, A-64
two-level decoder, A-64
Dynamically linked libraries (DLLs), 130–132
defined, 130
lazy procedure linkage version, 130

E

Early restart, 384*b*
Edge-triggered clocking methodology, 241, 242, A-47, A-72–A-73
advantage, A-48

www.yousefclass.ir

clocks, A-72–A-73
drawbacks, A-73–A-74
illustrated, A-49*f*
rising edge/falling edge, A-47
EDSAC (Electronic Delay Storage Automatic Calculator), 473.e2*f*, 473.e1, 54.e2
Eispack, 227.e2–227.e3, 227.e3
Electrically erasable programmable read-only memory (EEPROM), 373
Elements
combinational, 240
datapath, 243, 248–249
memory, A-49–A-57
state, 240, 242, 244*f*, A-47, A-49*b*
Embedded computers, 5–6
application requirements, 6
design, 5
growth, 54.e11
Embedded Microprocessor Benchmark Consortium (EEMBC), 54.e11
Embedded RISCs
See also Reduced instruction set computer (RISC) architectures
addressing modes, D-6
architecture summary, D-4*f*, D-4*f*
arithmetic/logical instructions, D-14*f*
conditional branches, D-16
constant extension summary, D-9*f*, D-9*f*
control instructions, D-15*f*
data transfer instructions, D-13*f*

www.youseficlass.ir

delayed branch and, D-23
DSP extensions, D-19
general purpose registers, D-5
instruction conventions, D-15*f*
instruction formats, D-8*f*
multiply-accumulate approaches, D-19*f*
Encoding
defined, C-31
RISC-V instruction, 85*f*, 119*f*
ROM control function, C-18
ROM logic function, A-15
x86 instruction, 153–154
ENIAC (Electronic Numerical Integrator and Calculator), 473.e1, 54.e2–54.e3, 54.e2, 54.e3
EPIC, 347.e4
Error correction, A-64–A-66
Error Detecting and Correcting Code (RAID 2), 458.e4
Error detection, A-65–A-66
Error detection code, 412
Ethernet, 23–24
EX stage
load instructions, 282*f*
overflow exception detection, 317, 320*f*
store instructions, 284*f*
Exabyte, 6*f*
Exception enable, 439*b*
Exceptions, 315–321
association, 321*b*

www.yousefclass.ir

datapath with controls for handling, 318*f*
defined, 193, 315
detecting, 315
event types and, 315
imprecise, 321*b*
interrupts *versus*, 315
pipelined computer example, 318*b*–319*b*
in pipelined implementation, 317–321
precise, 321*b*
reasons for, 316–317
result due to overflow in add instruction, 320*f*
in RISC-V architecture, 316–317
saving/restoring stage on, 440
Executable files
defined, 127–129
Execute or address calculation stage, 282
Execute/address calculation
control line, 291
load instruction, 282
store instruction, 282
Execution time
CPU, 32, 33–34
pipelining and, 276
as valid performance measure, 50–51
Explicit counters, C-23–C-24, C-26*f*
Exponents, 192

F

www.yousefclass.ir

Failures, synchronizer, A-75–A-76
Fallacies
See also Pitfalls
Amdahl's law, 548
arithmetic, 222
assembly language for performance, 158*b*
commercial binary compatibility importance, 158*b*
defined, 49
GPUs, B-72, B-75
low utilization uses little power, 50*b*
peak performance, 548*b*
pipelining, 345
powerful instructions mean higher performance, 157
right shift, 222*b*
False sharing, 457
Fast carry
with first level of abstraction, A-38–A-40
with "infinite" hardware, A-38
with second level of abstraction, A-40–A-45
Fast Fourier Transforms (FFT), B-53
Fault avoidance, 411
Fault forecasting, 411
Fault tolerance, 411
Fermi architecture, 515, 544
Field programmable devices (FPDs), A-77–A-78
Field programmable gate arrays (FPGAs), A-77
Fields
defined, 83

www.youseficlass.ir

format, C-31
names, 83
RISC-V, 83–89
Files, register, 244, 249, A-49*b*, A-53–A-55
Fine-grained multithreading, 506
Finite-state machines (FSMs), 449–454, A-66–A-71
control, C-8–C-22
controllers, 452*f*
for multicycle control, C-9*f*
for simple cache controller, 453–454
implementation, 451, A-69
Mealy, 452
Moore, 452*b*–453*b*
next-state function, 451, A-66
output function, A-66, A-68
state assignment, A-69
state register implementation, A-70*f*
style of, 452*b*–453*b*
synchronous, A-66
SystemVerilog, 459.e6*f*
traffic light example, A-67
Flash memory, 373
defined, 23
Flat address space, 469
Flip-flops
D flip-flops, A-50–A-51, A-52
defined, A-50–A-51
Floating point, 191–216

www.youseficlass.ir

assembly language, 207*f*
backward step, 227.e3–227.e4
binary to decimal conversion, 197*b*
branch, 206
challenges, 226
diversity *versus* portability, 227.e2–227.e3
division, 206
first dispute, 227.e1–227.e2
form, 192
fused multiply add, 214*b*
guard digits, 213*b*
history, 227.e2
IEEE 754 standard, 193–198
intermediate calculations, 212–213
operands, 207*f*
overflow, 192
packed format, 218
precision, 223
procedure with two-dimensional matrices, 80*b*
programs, compiling, 79*b*–80*b*
registers, 212*b*
representation, 192–193
RISC-V instruction frequency for, 226*f*
RISC-V instructions, 206–212
rounding, 212–213
sign and magnitude, 192
SSE2 architecture, 217, 217*f*
subtraction, 206

www.youseficlass.ir

underflow, 192
units, 213–214
in x86, 217*f*
Floating vectors, 227.e2
Floating-point addition, 198–201
arithmetic unit block diagram, 202*f*
binary, 199*b*–201*b*
illustrated, 200*f*
instructions, 206–212
steps, 198, 198, 198, 198–199
Floating-point arithmetic (GPUs), B-41–B-46
basic, B-42
double precision, B-45–B-46, B-74*b*
performance, B-44
specialized, B-42–B-44
supported formats, B-42
texture operations, B-44
Floating-point control and status register (fcsr), 193
Floating-point instructions
desktop RISC, D-12*f*
SPARC, D-31–D-32
Floating-point multiplication, 201–206
binary, 205*b*–206*b*
illustrated, 204*f*
instructions, 206
significands, 201–205
steps, 201–203, 203, 203, 203, 203–205
Flow-sensitive information, 144.e13*b*–144.e14*b*

www.youseficlass.ir

Flushing instructions, 308, 309–310
exceptions and, 319*b*
For loops, 142, 144.e25
inner, 144.e23
SIMD and, 553.e2
Format fields, C-31
Fortran, 162.e6
Forwarding, 294–307
ALU before, 299*f*
control, 300
datapath for hazard resolution, 302*f*
defined, 268–269
graphical representation, 269*f*
illustrations, 345.e20
multiple results and, 271
multiplexors, 300*f*
pipeline registers before, 299*f*
with two instructions, 268*b*–269*b*
Verilog implementation, 345.e3
Fractions, 192, 193
Frame buffer, 18
Frame pointers, 104–105
Front end, 144.e2
Fully associative caches
See also Caches
block replacement strategies, 445–446
choice of, 445
defined, 395

www.youseficlass.ir

memory block location, 395*f*
misses, 398
Fully connected networks, 527
Fused-multiply-add (FMA) operation, 214*b*, B-45

G

Game consoles, B-9
Gates, A-3–A-4, A-4–A-9
AND, A-12–A-13, C-7
delays, A-45
mapping ALU control function to, C-4–C-7
NAND, A-8–A-9
NOR, A-8–A-9, A-49*f*
Gather-scatter, 503, 544
General Purpose GPUs (GPGPUs), B-5
General-purpose registers, 147
architectures, 162.e2*f*
embedded RISCs, D-5
Generate
defined, A-39
example, A-44*b*–A-45*b*
super, A-40
Gigabyte, 6*f*
Global common subexpression elimination, 144.e5
Global memory, B-21, B-39
Global miss rates, 408*b*
Global optimization, 144.e4–144.e10
code, 144.e6

www.youseficlass.ir

implementing, 144.e7
Global pointers, 104*b*
GPU computing
See also Graphics processing units (GPUs)
defined, B-5–B-6
visual applications, B-6
GPU system architectures, B-7–B-12
graphics logical pipeline, B-10
heterogeneous, B-7–B-9
implications for, B-24–B-25
interfaces and drivers, B-9–B-10
unified, B-10–B-11
Graph coloring, 144.e11
Graphics displays
computer hardware support, 18
LCD, 18
Graphics logical pipeline, B-10
Graphics processing units (GPUs), 514–521
See also GPU computing
as accelerators, 514
attribute interpolation, B-43–B-44
defined, 46, 498–499, B-3
evolution, B-5
fallacies and pitfalls, B-72–B-75
floating-point arithmetic, B-16, B-41–B-46, B-74
GeForce 8-series generation, B-5
general computation, B-73*b*
General Purpose (GPGPUs), B-5

www.youseficlass.ir

graphics mode, B-6
graphics trends, B-4
history, B-3–B-4
logical graphics pipeline, B-13–B-14
mapping applications to, B-55–B-72
memory, 514
multilevel caches and, 514
N-body applications, B-65–B-68
NVIDIA architecture, 515–517
parallel memory system, B-36–B-41
parallelism, 515, B-76
performance doubling, B-4
perspective, 519–521
programming, B-12–B-25
programming interfaces to, B-17
real-time graphics, B-13
Graphics shader programs, B-14–B-15
Gresham's Law, 227, 227.e1
Grid computing, 525*b*–526*b*
Grids, B-19
GTX 280, 540–541
Guard digits
defined, 212–213
rounding with, 213*b*

H

Half precision, B-42
Halfwords, 112

www.youseficlass.ir

Hamming, Richard, 412
Hamming distance, 412
Hamming Error Correction Code (ECC), 412–413
  calculating, 412
Hard disks
  access times, 23
  defined, 23
Hardware
  as hierarchical layer, 13*f*
  language of, 14–16
  operations, 63–67
  supporting procedures in, 98–108
  synthesis, A-21
  translating microprograms to, C-28–C-32
  virtualizable, 418
Hardware description languages
  See also Verilog
  defined, A-20
  using, A-20–A-26
  VHDL, A-20–A-21
Hardware multithreading, 506–509
  coarse-grained, 506–507
  options, 507*f*
  simultaneous, 507
Hardware-based speculation, 331–333
Harvard architecture, 54.e3
Hazard detection units, 303
  pipeline connections for, 306–307

www.yousefclass.ir

Hazards
See also Pipelining
control, 271–274, 307–315
data, 268–271, 294–307
forwarding and, 302*b*
structural, 267–268, 284
Heap
allocating space on, 104–105
defined, 105
Heterogeneous systems, B-4–B-5
architecture, B-7–B-12
defined, B-3
Hexadecimal numbers, 82
binary number conversion to, 82*f*, 83*b*
Hierarchy of memories, 12
High-level languages, 14–16
benefits, 16
computer architectures, 162.e4
importance, 16
High-level optimizations, 144.e3–144.e4
Hit rate, 368
Hit time
cache performance and, 393–394
defined, 368–369
Hit under miss, 460
Hold time, A-52–A-53
Horizontal microcode, C-32
Hot-swapping, 458.e6–458.e7

www.yousefclass.ir

Human genome project, 4

I

I/O, 529.e1–529.e2, 529.e2, 529.e2
  on system performance, 458.e1*b*–458.e2*b*
I/O benchmarks, *See* Benchmarks
IBM 360/85, 473.e5
IBM 701, 54.e4
IBM 7030, 347.e1
IBM ALOG, 227.e6
IBM Blue Gene, 553.e8, 553.e8–553.e9
IBM Personal Computer, 162.e5, 54.e7
IBM System/360 computers, 54.e5*f*, 347.e1, 227.e5, 227.e6
IBM z/VM, 473.e12
ID stage
  branch execution in, 309, 310
  load instructions, 282*f*
  store instruction in, 281*f*
IEEE 754 floating-point standard, 193–198, 194*f*, 227.e7–227.e9
  See also Floating point
  first chips, 227.e7–227.e9
  in GPU arithmetic, B-42
  implementation, 227.e9
  rounding modes, 213–214
  today, 227.e9
If statements, 115–116
If-then-else, 93*b*
Imagination Technologies, 145

www.yousefclass.ir

Immediate addressing, 118
Immediate instructions, 72
Imprecise interrupts, 321*b*, 347.e2–347.e3
Index-out-of-bounds check, 96
Induction variable elimination, 144.e6
Inheritance, 144.e14
In-order commit, 330–331
Input devices, 16–17
Inputs, 253
Instances, 144.e14
Instruction count, 36, 38
Instruction decode/register file read stage
  control line, 290–294
  load instruction, 279
  store instruction, 284
Instruction execution illustrations, 345.e13–345.e20
  clock cycle 9, 345.e25*f*
  clock cycles 1 and 2, 345.e21*f*
  clock cycles 3 and 4, 345.e22*f*
  clock cycles 5 and 6, 345.e23*f*
  clock cycles 7 and 8, 345.e24*f*
  examples, 345.e15–345.e20
  forwarding, 345.e20, 345.e20
  no hazard, 345.e15
  pipelines with stalls and forwarding, 345.e20
Instruction fetch stage
  control line, 291
  load instruction, 279

www.youseficlass.ir

store instruction, 284
Instruction formats, 153
defined, 82
desktop/server RISC architectures, D-7*f*
embedded RISC architectures, D-8*f*
I-type, 84
MIPS, 146*f*
RISC-V, 146*f*
R-type, 84, 253–254
SB-type, 115
S-type, 84, 84–85
UJ-type, 115, 115
U-type, 113–114
x86, 153–154
Instruction latency, 346–347
Instruction mix, 39–40, 54.e9
Instruction set architecture
branch address calculation, 246
defined, 22, 52
history, 162
maintaining, 52
protection and, 419
thread, B-31–B-34
virtual machine support, 418
Instruction sets, B-49
MIPS-32, 146*f*
RISC-V, 160
x86 growth, 162*f*

www.yousefclass.ir

Instruction-level parallelism (ILP), 344–345
See also Parallelism
compiler exploitation, 347.e4–347.e5
defined, 43*b*, 321–322
exploitation, increasing, 333
and matrix multiply, 342–345
Instructions, 60, D-25–D-27, D-40, D-40–D-43
See also Arithmetic instructions, MIPS, Operands
add immediate, 72–74
addition, 176
Alpha, D-27–D-29
arithmetic-logical, 243–244
ARM, D-36–D-38
assembly, 65
basic block, 95*b*
cache-aware, 472
conditional branch, 92–93, 93*b*
conditional move, 313*b*–314*b*
data transfer, 68
decision-making, 92–98
defined, 14, 62
desktop RISC conventions, D-12*f*
as electronic signals, 81–82
embedded RISC conventions, D-15*f*
encoding, 85*f*
fetching, 245*f*
floating-point, 206–212
floating-point (x86), 217*f*

www.youseficlass.ir

flushing, 308, 309–310
immediate, 72
introduction to, 62–63
left-to-right flow, 277
load, 69
logical operations, 89–92
M32R, D-40
memory access, B-33–B-34
memory-reference, 237
multiplication, 183
nop, 304–305
PA-RISC, D-34–D-36
performance, 35–36
pipeline sequence, 304*f*
PowerPC, D-12–D-13, D-32–D-34
PTX, B-31, B-32*f*
representation in computer, 81–89
restartable, 440–441
resuming, 440*b*–441*b*
R-type, 243–244, 248–249
SPARC, D-29–D-32
store, 71
store-conditional doubleword, 122–123
subtraction, 176
SuperH, D-39–D-40
thread, B-30–B-31
Thumb, D-38–D-39
vector, 500–502

www.youseficlass.ir

as words, 62
x86, 146–155
Instructions per clock cycle (IPC), 322
Integrated circuits (ICs), 19
See also specific chips
cost, 27
defined, 25
manufacturing process, 26
very large-scale (VLSIs), 25
Intel Core i7, 46–49, 236, 493, 540–545
address translation for, 460*f*
architectural registers, 337–338
caches in, 461*f*
memory hierarchies of, 459–464
microarchitecture, 337
performance of, 462
SPEC CPU benchmark, 46–48
SPEC power benchmark, 48–49
TLB hardware for, 460*f*
Intel Core i7 920, 337–340
microarchitecture, 337
Intel Core i7 960
benchmarking and rooflines of, 540–545
Intel Core i7 Pipelines, 334–342, 337–340
memory components, 338*f*
performance, 340–342
program performance, 341*b*
specification, 335*f*

www.youseficlass.ir

Intel IA-64 architecture, 162.e2*f*
Intel Paragon, 553.e6–553.e7
Intel Threading Building Blocks, B-60
Intel x86 microprocessors
  clock rate and power for, 40*f*
Interference graphs, 144.e10
Interleaving, 390
Interprocedural analysis, 144.e13*b*–144.e14*b*
Interrupt enable, 439*b*
Interrupt-driven I/O, 529.e3
Interrupts
  defined, 193, 315
  event types and, 315
  exceptions *versus*, 315
  imprecise, 321*b*, 347.e2–347.e3
  precise, 321*b*
  vectored, 316
Intrinsity FastMATH processor, 387–389
  caches, 388*f*
  data miss rates, 389*f*, 399*f*
  read processing, 434*f*
  TLB, 432–435
  write-through processing, 434*f*
Inverted page tables, 429
Issue packets, 324–325
I-type, 87*b*

J

www.youseficlass.ir

Java
bytecode, 132
bytecode architecture, 144.e10–144.e12
characters in, 111–113
compiling in, 144.e18–144.e19
goals, 132
interpreting, 132, 144–145, 144.e14
keywords, 144.e20
method invocation in, 144.e20
pointers, 144.e25–144.e26
primitive types, 144.e25
programs, starting, 132–133
reference types, 144.e25
sort algorithms, 141*f*
strings in, 111–113
translation hierarchy, 132*f*
while loop compilation in, 144.e17*b*–144.e18*b*
Java Virtual Machine (JVM), 145, 144.e15
Jump-and-link register instruction (jalr), 97–98, 99
Jump instructions, D-26
branch instruction *versus*, 250*f*
control and datapath for, 251
implementing, 237–240
instruction format, 250
Just In Time (JIT) compilers, 133, 552

K

Karnaugh maps, A-18

www.youseficlass.ir

Kernel mode, 437
Kernels
CUDA, B-19, B-24
defined, B-19–B-22
Kilobyte, 6*f*

L

LAPACK, 223–224
Large-scale multiprocessors, 553.e6, 553.e6–553.e7
Latches
D latch, A-50–A-51, A-51
defined, A-50–A-51
Latency
instruction, 346–347
memory, B-74*b*
pipeline, 276*b*
use, 325–327
lb (load byte), 64*f*
lbu (load byte, unsigned), 64*f*
ld (load doubleword), 64*f*
Leaf procedures
See also Procedures
defined, 102
example, 112*f*
Least recently used (LRU)
as block replacement strategy, 445–446
defined, 401
pages, 426–428

www.yousefclass.ir

Least significant bits
defined, 74
SPARC, D-31
Left-to-right instruction flow, 277
Level-sensitive clocking, A-73–A-74, A-74–A-75
defined, A-73–A-74
two-phase, A-74
lh (load halfword), 64*f*
lhu (load halfword, unsigned), 64*f*
Link, 529.e1–529.e2
Linkers, 127–129
defined, 127
executable files, 127–129
steps, 127
Linking object files, 128*b*–129*b*
Linpack, 530, 227.e2–227.e3, 227.e3
Liquid crystal displays (LCDs), 18
LISP, SPARC support, D-30
Live range, 144.e10
Livermore Loops, 54.e10
Load balancing, 497*b*–498*b*
Load byte, 109
Load doubleword, 69, 71–72
Load instructions
See also Store instructions
access, B-41
base register, 254
compiling with, 71*b*

www.youseficlass.ir

datapath in operation for, 259*f*
defined, 69
EX stage, 282*f*
halfword unsigned, 112
ID stage, 281*f*
IF stage, 281*f*
load byte unsigned, 78
load half, 112
MEM stage, 283*f*
pipelined datapath in, 286*f*
signed, 78*b*
unit for implementing, 247*f*
unsigned, 78*b*
WB stage, 283*f*
Loaders, 130
Load-reserved doubleword, 122–123
Load-store architectures, 162.e2
Load upper immediate, 113–114
Load-use data hazard, 269–271, 308
Load-use stalls, 308
Load word, 113*b*
Load word unsigned, 113*b*
Local area networks (LANs), 24
See also Networks
Local memory, B-21, B-40
Local miss rates, 408*b*
Local optimization, 144.e4
See also Optimization

www.youseficlass.ir

implementing, 144.e7
Locality
principle, 366–367
spatial, 366, 369*b*
temporal, 366, 369*b*
Lock synchronization, 121
Locks, 510–513
Logic
address select, C-24, C-25*f*
ALU control, C-6–C-7
combinational, 242, A-5, A-9–A-20
components, 241
control unit equations, C-11*f*
design, 240–243
equations, A-7*b*
minimization, A-18
programmable array (PAL), A-77
sequential, A-4–A-5, A-55–A-57
two-level, A-11–A-14
Logical operations, 89–92
AND, 90
desktop RISC, D-11*f*, D-11*f*
embedded RISC, D-13*f*
NOT, 91
OR, 91
shifts, 90
xor, 91
Long instruction word (LIW), 347.e4

www.youseficlass.ir

www.yousefclass.ir

Lookup tables (LUTs), A-77–A-78
Loop unrolling
defined, 327–328, 144.e3–144.e4
for multiple-issue pipelines, 327*b*–328*b*
register renaming and, 327
Loops, 94–96
conditional branches in, 115–116
for, 142
prediction and, 312*b*
test, 142, 143
while, compiling, 94*b*–95*b*
lr.d (load reserved), 64*f*
lui (load upper immediate), 64*f*
lw (load word), 64*f*
lwu (load word, unsigned), 64*f*

M

M32R, D-15, D-40
Machine code, 82
Machine instructions, 82
Machine language, 15*f*
branch offset in, 116*b*–117*b*
decoding, 118–120
defined, 14, 82
illustrated, 15*f*
RISC-V, 87–89
SRAM, 19–22
translating RISC-V assembly language into, 85*b*–86*b*

www.youseficlass.ir

Main memory, 420
See also Memory
defined, 23
page tables, 429
physical addresses, 420
Mapping applications, B-55–B-72
Mark computers, 54.e3
Matrix multiply, 218–222, 545–548
Mealy machine, 452, A-67, A-70–A-71, A-71*b*
Mean time to failure (MTTF), 410–411
versus AFR of disks, 410*b*–411*b*
improving, 411–412
Media Access Control (MAC) address, 529.e6
Megabyte, 6*f*
Memory
addresses, 78*b*
affinity, 538*f*
atomic, B-21
bandwidth, 371–372, 389*b*
cache, 19–22, 375–390, 390–410
CAM, 400*b*–401*b*
constant, B-40
control, C-26
defined, 19
DRAM, 19, 371–373, A-62–A-64
flash, 23
global, B-21, B-39
GPU, 514

www.youseficlass.ir

instructions, datapath for, 247
local, B-21, B-40
main, 23
nonvolatile, 22–23
operands, 68–72
parallel system, B-36–B-41
read-only (ROM), A-14–A-16
SDRAM, 371–372
secondary, 23
shared, B-17, B-39–B-40
spaces, B-39
SRAM, A-57–A-59
stalls, 392
technologies for building, 24–28
texture, B-40
virtual, 419–443
volatile, 22–23
Memory access instructions, B-33–B-34
Memory access stage
control line, 292*f*
load instruction, 282*f*
store instruction, 282
Memory bandwidth, 540–541, 549*b*
Memory consistency model, 458*b*
Memory elements, A-49–A-57
clocked, A-50
D flip-flop, A-50–A-51, A-52
D latch, A-51

www.youseficlass.ir

DRAMs, A-62–A-64
flip-flop, A-50
hold time, A-52–A-53
latch, A-50
setup time, A-52–A-53, A-53*f*
SRAMs, A-57–A-59
unclocked, A-50
Memory hierarchies, 537
of ARM cortex-A53, 459–464
block (or line), 367–368
cache performance, 390–410
caches, 375–390
common framework, 443–449
defined, 367
design challenges, 449*b*
development, 473.e5–473.e7
exploiting, 364
of Intel Core i7, 459–464
level pairs, 368*f*
multiple levels, 367
overall operation of, 435*b*–436*b*
parallelism and, 454–458, 458.e1–458.e2
pitfalls, 468–472
program execution time and, 409
quantitative design parameters, 443*f*
redundant arrays and inexpensive disks, 458
reliance on, 369
structure, 367*f*

www.youseficlass.ir

structure diagram, 370*f*
variance, 409*b*
virtual memory, 419–443
Memory rank, 373
Memory technologies, 370–375
disk memory, 373–375
DRAM technology, 370, 371–373
flash memory, 373
SRAM technology, 370, 371
Memory-mapped I/O, 529.e2
Memory-stall clock cycles, 391
Message passing
defined, 521
multiprocessors, 521–526
Metastability, A-75–A-76
Methods
defined, 144.e14
invoking in Java, 144.e19–144.e20
Microarchitectures, 337
Intel Core i7 920, 337–340
Microcode
assembler, C-30
control unit as, C-28*f*
defined, C-27
dispatch ROMs, C-30, C-30*f*
horizontal, C-32
vertical, C-32
Microinstructions, C-31

www.yousefclass.ir

Microprocessors
design shift, 493
multicore, 8, 43, 492–493
Microprograms
as abstract control representation, C-30–C-31
field translation, C-28–C-29
translating to hardware, C-28–C-32
Migration, 456
Million instructions per second (MIPS), 51
Minterms
defined, A-12–A-13, C-20
in PLA implementation, C-20
MIP-map, B-44
MIPS and RISC-V
common features between, 145
MIPS-16
16-bit instruction set, D-41–D-42
immediate fields, D-41
instructions, D-40–D-43
MIPS core instruction changes, D-42–D-43
PC-relative addressing, D-41
MIPS-32 instruction set, 145
MIPS-64 instructions, 145, D-25–D-27
conditional procedure call instructions, D-27
constant shift amount, D-25
jump/call not PC-relative, D-26
move to/from control registers, D-26
nonaligned data transfers, D-25

www.youseficlass.ir

NOR, D-25
parallel single precision floating-point operations, D-27
reciprocal and reciprocal square root, D-27
SYSCALL, D-25
TLB instructions, D-26–D-27
Mirroring, 458.e4
Miss penalty
defined, 368–369
determination, 383–384
multilevel caches, reducing, 402–405
Miss rates
block size *versus*, 383–384
data cache, 444*f*
defined, 368
global, 408*b*
improvement, 383–384
Intrinsity FastMATH processor, 389
local, 408*b*
miss sources, 448
split cache, 389*b*
Miss under miss, 460
MMX (MultiMedia eXtension), 217
Moore machines, 452, A-67, A-70–A-71, A-71*b*
Moore's law, 11, 371, 514, 529.e1–529.e2, B-72*b*
Most significant bit
1-bit ALU for, A-33*f*
defined, 74
MS-DOS, 473.e15, 473.e15

www.youseficlass.ir

Multicore, 509–514
Multicore multiprocessors, 8, 43
defined, 8, 492–493
MULTICS (Multiplexed Information and Computing Service), 473.e8
Multilevel caches
See also Caches
complications, 408*b*
defined, 390, 408*b*
miss penalty, reducing, 402–405
performance of, 402*b*–403*b*
summary, 409–410
Multimedia extensions
desktop/server RISCs, D-16–D-18
as SIMD extensions to instruction sets, 553.e3
vector *versus*, 501*b*–502*b*
Multiple dimension arrays, 212
Multiple instruction multiple data (MIMD), 550–551
defined, 499, 500
first multiprocessor, 553.e3–553.e4
Multiple instruction single data (MISD), 499–500
Multiple issue, 322
code scheduling, 326*b*–327*b*
dynamic, 322, 328–333
issue packets, 324–325
loop unrolling and, 327*b*–328*b*
processors, 322
static, 322, 324–328

www.youseficlass.ir

throughput and, 332*b*
Multiple processors, 545–548
Multiple-clock-cycle pipeline diagrams, 286–287
five instructions, 288*f*
illustrated, 287–290
Multiplexors, A-10
controls, 451
in datapath, 255*f*
defined, 237–238
forwarding, control values, 300*f*
selector control, 251
two-input, A-10
Multiplicand, 178
Multiplication, 177–183
See also Arithmetic
fast, hardware, 182
faster, 182–183
first algorithm, 180*f*
floating-point, 201–206
hardware, 178–182
instructions, 183
operands, 183
product, 183
sequential version, 178–182
signed, 182
Multiplier, 178
Multiply algorithm, 178–182
Multiply-add (MAD), B-42

www.youseficlass.ir

Multiprocessors
benchmarks, 530–540
bus-based coherent, 553.e6
defined, 492
historical perspective, 553
large-scale, 553.e6, 553.e6–553.e7
message-passing, 521–526
multithreaded architecture, B-26–B-27, B-36
organization, 491, 521
for performance, 549
shared memory, 492–493, 509–514
software, 493*f*
TFLOPS, 553.e5
UMA, 510
Multistage networks, 527–528
Multithreaded multiprocessor architecture, B-25–B-36
conclusion, B-36
ISA, B-31–B-34
massive multithreading, B-25–B-26
multiprocessor, B-26–B-27
multiprocessor comparison, B-35–B-36
SIMT, B-27–B-29
special function units (SFUs), B-35
streaming processor (SP), B-34
thread instructions, B-30–B-31
threads/thread blocks management, B-30
Multithreading, B-25–B-26
coarse-grained, 506–507

www.youseficlass.ir

defined, 498–499
fine-grained, 506
hardware, 506–509
simultaneous (SMT), 507
Must-information, 144.e13*b*–144.e14*b*
Mutual exclusion, 121

N

Name dependence, 327
NAND gates, A-8–A-9
NAS (NASA Advanced Supercomputing), 532
N-body
all-pairs algorithm, B-65
GPU simulation, B-71
mathematics, B-65–B-66
multiple threads per body, B-68–B-72
optimization, B-67
performance comparison, B-69–B-70
results, B-70–B-72
shared memory use, B-67–B-68
Negation shortcut, 78–79
Nested procedures, 102–104
compiling recursive procedure showing, 102*b*–103*b*
NetFPGA 10-Gigagit Ethernet card, 529.e1*f*, 529.e2*f*
Network of Workstations, 553.e7–553.e8
Network topologies, 526–529
implementing, 528–529
multistage, 529*f*

www.youseficlass.ir

Networking, 529.e3–529.e4
operating system in, 529.e3–529.e5
performance improvement, 529.e6–529.e9
Networks, 23–24
advantages, 23
bandwidth, 527
crossbar, 527–528
fully connected, 527
local area (LANs), 23–24
multistage, 527–528
wide area (WANs), 23–24
Newton's iteration, 212*b*
Next state
nonsequential, C-24
sequential, C-23–C-24
Next-state function, 451, A-66
defined, 451
implementing, with sequencer, C-22–C-28
Next-state outputs, C-27, C-12*b*–C-13*b*
example, C-12
implementation, C-12–C-13
logic equations, C-12*b*–C-13*b*
truth tables, C-13–C-15
No Redundancy (RAID 0), 458.e3
No write allocation, 386
Nonblocking assignment, A-24
Nonblocking caches, 334*b*, 460
Nonuniform memory access (NUMA), 510

www.youseficlass.ir

Nonvolatile memory, 22–23
Nops, 304–305
NOR gates, A-8–A-9
  cross-coupled, A-49*f*
  D latch implemented with, A-51*f*
NOR operation, D-25
NOT operation, 91, A-6
Numbers
  binary, 74
  computer *versus* real-world, 215
  decimal, 74, 77*b*
  denormalized, 216
  hexadecimal, 83
  signed, 74–81
  unsigned, 74–81
NVIDIA GeForce 8800, B-46–B-55
  all-pairs N-body algorithm, B-71
  dense linear algebra computations, B-51–B-53
  FFT performance, B-53
  instruction set, B-49
  performance, B-51
  rasterization, B-50
  ROP, B-50–B-51
  scalability, B-51
  sorting performance, B-54–B-55
  special function approximation statistics, B-43*f*
  special function unit (SFU), B-50
  streaming multiprocessor (SM), B-48–B-49

www.youseficlass.ir

streaming processor, B-49–B-50
streaming processor array (SPA), B-46
texture/processor cluster (TPC), B-47
NVIDIA GPU architecture, 515–517
NVIDIA GTX 280, 541*f*, 542*f*
NVIDIA Tesla GPU, 540–545

O

Object files, 128*b*–129*b*
debugging information, 127
header, 126
linking, 128*b*–129*b*
relocation information, 126
static data segment, 126
symbol table, 127
text segment, 126
Object-oriented languages
See also Java
brief history, 162.e7
defined, 145, 144.e14
One's complement, 81, A-29
Opcodes
control line setting and, 256
defined, 83, 254
OpenGL, B-13
OpenMP (Open MultiProcessing), 512*b*–513*b*, 532
Operands, 67–74
See also Instructions

www.yousefclass.ir

32-bit immediate, 113–114
adding, 175
arithmetic instructions, 67
compiling assignment when in memory, 69*b*
constant, 72–74
division, 183–191
floating-point, 207*f*
memory, 68–72
multiplication, 177–183
RISC-V, 64*f*
Operating systems
brief history, 473.e8
defined, 13
encapsulation, 22
in networking, 529.e3–529.e5
Operations
atomic, implementing, 122
hardware, 63–67
logical, 89–92
x86 integer, 151–152
Optimization
class explanation, 144.e13*f*
compiler, 141*f*
control implementation, C-27
global, 144.e4–144.e10
high-level, 144.e3–144.e4
local, 144.e4–144.e10, 144.e7
manual, 144

www.yousefclass.ir

or (inclusive or), 64*f*
OR operation, 176, A-6
ori (inclusive or immediate), 64*f*
Out-of-order execution
  defined, 330
  performance complexity, 408*b*–409*b*
  processors, 334*b*
Output devices, 16–17
Overflow
  defined, 75, 192
  detection, 176
  exceptions, 318*f*
  floating-point, 193
  occurrence, 175
  saturation and, 177*b*
  subtraction, 175

P

P+Q redundancy (RAID 6), 458.e6
Packed floating-point format, 218
Page faults, 426
  See also Virtual memory
  for data access, 461
  defined, 420–421
  handling, 422, 439–441
  virtual address causing, 432–435
Page tables, 445
  defined, 424–425

www.yousefclass.ir

illustrated, 427*f*
indexing, 424–425
inverted, 429
levels, 429
main memory, 429
register, 424–425
storage reduction techniques, 429
updating, 424
VMM, 441*b*
Pages
See also Virtual memory
defined, 420–421
dirty, 430*b*
finding, 424–425
LRU, 426–428
offset, 421
physical number, 421
placing, 424–425
size, 422*f*
virtual number, 421
Parallel bus, 529.e1–529.e2
Parallel execution, 121
Parallel memory system, B-36–B-41
See also Graphics processing units (GPUs)
caches, B-38
constant memory, B-40
DRAM considerations, B-37–B-38
global memory, B-39

www.yousefclass.ir

load/store access, B-41
local memory, B-40
memory spaces, B-39
MMU, B-38–B-39
ROP, B-41
shared memory, B-39–B-40
surfaces, B-41
texture memory, B-40
Parallel processing programs, 494–499
creation difficulty, 494–499
defined, 492
for message passing, 511*b*–513*b*
great debates in, 553.e4–553.e6
for shared address space, 511*b*–513*b*
use of, 549
Parallel reduction, B-62
Parallel scan, B-60–B-63
CUDA template, B-61*f*
inclusive, B-60
tree-based, B-62*f*
Parallel software, 493
Parallelism, 12, 43*b*, 321–334
and computers arithmetic, 216–217
data-level, 226, 500
debates, 553.e4–553.e6
GPUs and, 514, B-76
instruction-level, 43, 321–322, 333
memory hierarchies and, 454–458, 458.e1–458.e2

www.yousefclass.ir

multicore and, 509*b*
multiple issue, 322*b*
multithreading and, 507
performance benefits, 44
process-level, 492
redundant arrays and inexpensive disks, 458
subword, D-17
task, B-24
task-level, 492
thread, B-22
Paravirtualization, 472
PA-RISC, D-14, D-17
branch vectored, D-35
conditional branches, D-34, D-35*f*
debug instructions, D-36
decimal operations, D-35
extract and deposit, D-35
instructions, D-34–D-36
load and clear instructions, D-36
multiply/add and multiply/subtract, D-36
nullification, D-34
nullifying branch option, D-25
store bytes short, D-36
synthesized multiply and divide, D-34–D-35
Parity, 458.e4
bits, 412–413
code, 420, A-64–A-65
PARSEC (Princeton Application Repository for Shared Memory

www.youseficlass.ir

Computers), 532
Pass transistor, A-62*b*–A-64*b*
PCI-Express (PCIe), 529, B-7–B-8, 529.e1–529.e2
PC-relative addressing, 115–116, 118
Peak floating-point performance, 534
Pentium bug morality play, 224*f*
Performance, 28–40
assessing, 28
classic CPU equation, 36–40
components, 38*f*
CPU, 33–35
defining, 29–32
equation, using, 36–40
improving, 34*b*–35*b*
instruction, 35–36
measuring, 32–33, 54.e9
program, 9–10
ratio, 31
relative, 31*b*
response time, 30*b*
sorting, B-49–B-50
throughput, 30*b*
time measurement, 32
Personal computers (PCs), 7*f*
defined, 5
Personal mobile device (PMD)
defined, 6–7
Petabyte, 6*f*

www.yousefclass.ir

Physical addresses, 420
mapping to, 420–421
space, 509, 511*b*–513*b*
Physically addressed caches, 436–437
Pipeline registers
before forwarding, 298–300
dependences, 297–298, 298*f*
forwarding unit selection, 302
Pipeline stalls, 270
avoiding with code reordering, 270*b*–271*b*
data hazards and, 303–307
insertion, 305*f*
load-use, 308
as solution to control hazards, 272*f*
Pipelined branches, 310*b*
Pipelined control, 290–294
See also Control
control lines, 290–291, 291
overview illustration, 306*f*
specifying, 291
Pipelined datapaths, 276–294
with connected control signals, 294*f*
with control signals, 290–294
corrected, 286*f*
illustrated, 279*f*
in load instruction stages, 286*f*
Pipelined dependencies, 296*f*
Pipelines

www.youseficlass.ir

branch instruction impact, 308*f*
effectiveness, improving, 347.e3–347.e4
execute and address calculation stage, 280, 282
five-stage, 264, 280, 288*b*–290*b*
graphic representation, 269*f*, 286–290
instruction decode and register file read stage, 278*f*, 282
instruction fetch stage, 279*f*, 282
instructions sequence, 304*f*
latency, 276*b*
memory access stage, 280, 282
multiple-clock-cycle diagrams, 286–287
performance bottlenecks, 332–333
single-clock-cycle diagrams, 286–287
stages, 264–265
static two-issue, 325*f*
write-back stage, 279, 284
Pipelining, 12, 262–276
advanced, 333–334
benefits, 262
control hazards, 271–274
data hazards, 268–271
exceptions and, 317–321
execution time and, 276*b*
fallacies, 345–346
hazards, 267–271
instruction set design for, 267
laundry analogy, 263*f*
overview, 262–276

www.youseficlass.ir

paradox, 263–264
performance improvement, 267
pitfall, 345–346
simultaneous executing instructions, 276*b*
speed-up formula, 265
structural hazards, 267–268, 284
summary, 314–315
throughput and, 276*b*
Pitfalls
See also Fallacies
address space extension, 384–385
arithmetic, 222–225
associativity, 469*b*
defined, 49
GPUs, B-74
ignoring memory system behavior, 468*b*
memory hierarchies, 468–472
out-of-order processor evaluation, 469*b*
performance equation subset, 50*b*
pipelining, 345–346
pointer to automatic variables, 159*b*
sequential word addresses, 159*b*
simulating cache, 468
software development with multiprocessors, 548*b*
VMM implementation, 470–472
Pixel shader example, B-15–B-17
Pixels, 18
Pointers

www.youseficlass.ir

arrays *versus*, 141–144
frame, 104–105
global, 104*b*
incrementing, 143
Java, 144.e25–144.e26
stack, 99, 102–104
Polling, 529.e6
Pop, 99
Power
clock rate and, 40
critical nature of, 53
efficiency, 333–334
relative, 41*b*–42*b*
PowerPC
algebraic right shift, D-33
branch registers, D-32–D-33
condition codes, D-12–D-13
instructions, D-12–D-13
instructions unique to, D-32–D-34
load multiple/store multiple, D-33
logical shifted immediate, D-33
rotate with mask, D-33
Precise interrupts, 321*b*
Prediction, 12
2-bit scheme, 312
accuracy, 312
dynamic branch, 310–314
loops and, 312*b*

www.youseficlass.ir

steady-state, 312
Prefetching, 472, 536
Primitive types, 144.e25
Procedure calls
preservation across, 104
Procedures, 98–108
compiling, 100*b*–101*b*
compiling, showing nested procedure linking, 100*b*–101*b*
execution steps, 98
frames, 104
leaf, 102
nested, 102*b*–103*b*
recursive, 107*b*
for setting arrays to zero, 141*f*
sort, 135–140
strcpy, 110*b*–111*b*
string copy, 110*b*–111*b*
swap, 134–135
Process identifiers, 438
Process-level parallelism, 492
Processors, 234
control, 19
as cores, 43
datapath, 19
defined, 17*b*, 19
dynamic multiple-issue, 322
multiple-issue, 322
out-of-order execution, 334*b*, 408*b*–409*b*

www.youseficlass.ir

performance growth, 44*f*
ROP, B-12, B-41
speculation, 323–324
static multiple-issue, 322, 324–328
streaming, B-34
superscalar, 328, 507–508, 347.e4
technologies for building, 24–28
two-issue, 325–327
vector, 499–500
VLIW, 324
Product, 178
Product of sums, A-11
Program counters (PCs), 243
changing with conditional branch, 313*b*–314*b*
defined, 99, 243
exception, 437, 439
incrementing, 243, 245*f*
instruction updates, 279
Program performance
elements affecting, 39*t*
understanding, 9
Programmable array logic (PAL), A-77
Programmable logic arrays (PLAs)
component dots illustration, A-16*f*
control function implementation, C-7*f*, C-20
defined, A-12–A-13
example, A-13*b*–A-14*b*
illustrated, A-13*f*

www.youseficlass.ir

ROMs and, A-15–A-16
size, C-20
truth table implementation, A-13
Programmable logic devices (PLDs), A-77
Programmable ROMs (PROMs), A-14
Programming languages
See also specific languages
brief history of, 162.e6–162.e7
object-oriented, 145
variables, 67
Programs
assembly language, 125
Java, starting, 132–133
parallel processing, 492
starting, 124–133
translating, 124–133
Propagate
defined, A-39
example, A-44*b*–A-45*b*
super, A-40
Protected keywords, 144.e20
Protection
defined, 420
implementing, 437–439
mechanisms, 473.e12
VMs for, 416–417
Protection group, 458.e4
Pseudoinstructions

www.yousefclass.ir

defined, 125
summary, 126
Pthreads (POSIX threads), 532
PTX instructions, B-31, B-32*f*
Public keywords, 144.e20
Push
defined, 99
using, 102–104

Q

Quad words, 151
Quicksort, 403*b*–405*b*, 404*f*
Quotient, 184

R

Race, A-72–A-73
Radix sort, 403*b*–405*b*, 404*f*, B-63–B-65
CUDA code, B-64*f*
implementation, B-63–B-65
RAID, *See* Redundant arrays of inexpensive disks (RAID)
RAM, 9
Raster operation (ROP) processors, B-12, B-41, B-50–B-51
fixed function, B-41
Raster refresh buffer, 18
Rasterization, B-50
Ray casting (RC), 544
Read-only memories (ROMs), A-14–A-16
control entries, C-16*b*–C-18*b*

www.yousefclass.ir

control function encoding, C-19
dispatch, C-25*f*
implementation, C-15–C-19
logic function encoding, A-15
overhead, C-18
PLAs and, A-15–A-16, A-16
programmable (PROM), A-14
total size, C-15–C-16
Read-stall cycles, 391
Read-write head, 373
Receive message routine, 521
Recursive procedures, 107*b*
See also Procedures
clone invocation, 102
Reduced instruction set computer (RISC) architectures, D-3–D-5, D-5–D-9, D-9–D-16, D-16–D-18, D-19, D-20–D-25, D-25–D-27, D-27–D-29, D-29–D-32, D-32–D-34, D-34–D-36, D-36–D-38, D-38–D-39, D-39–D-40, D-40, D-40–D-43, D-43–D-45, 347.e3, 162.e4
See also Desktop and server RISCs, Embedded RISCs
group types, D-3–D-4
instruction set lineage, D-44*f*
Reduction, 511
Redundant arrays of inexpensive disks (RAID), 458.e1–458.e2
history, 458.e6–458.e7
RAID 0, 458.e3
RAID 1, 458.e4
RAID 2, 458.e4
RAID 3, 458.e4
RAID 4, 458.e4–458.e5

www.youseficlass.ir

RAID 5, 458.e5–458.e6
RAID 6, 458.e6
spread of, 458.e5
summary, 458.e6–458.e7
use statistics, 458.e6*f*
Reference bit, 428*b*
References
absolute, 127
types, 144.e25
Register addressing, 118*f*
Register allocation, 144.e10–144.e12
Register files, A-49*b*, A-53–A-55
in behavioral Verilog, A-56
defined, 244, A-49*b*, A-53
single, 249, 249
two read ports implementation, A-54*f*
with two read ports/one write port, A-54*f*
write port implementation, A-55*f*
Register-memory architecture, 162.e2
Registers, 148, 149–151
architectural, 316, 337–338
base, 69
clock cycle time and, 67
compiling C assignment with, 67*b*–68*b*
defined, 67
destination, 254
floating-point, 212*b*
left half, 280

www.yousefclass.ir

number specification, 244
page table, 424–425
pipeline, 297–298, 297–298, 298*f*, 302
primitives, 67
renaming, 327
right half, 280
RISC-V conventions, 255*f*
spilling, 71
Status, 316
temporary, 68, 100
variables, 68
Relative performance, 31*b*
Relative power, 41*b*–42*b*
Reliability, 410–411
Remainder, defined, 184
Reorder buffers, 334*b*
Replication, 456
Requested word first, 384
Request-level parallelism, 524
Reservation stations
buffering operands in, 329
defined, 329
Response time, 30*b*
Restartable instructions, 440–441
Return address, 99
R-format
ALU operations, 245*f*
Ripple carry

www.youseficlass.ir

adder, A-29
carry lookahead speed *versus*, A-45*b*
RISC-V, 62, 85–87
architecture, 190*f*
arithmetic instructions, 63
arithmetic/logical instructions not in, D-21*f*, D-23*f*
assembly instruction, mapping, 81*b*–82*b*
common extensions to, D-20–D-25
compiling C assignment statements into, 65*b*
compiling complex C assignment into, 66*b*
control instructions not in, D-21*f*
control registers, 439*b*
control unit, C-10
data transfer instructions not in, D-20*f*, D-22*f*
divide in, 189
exceptions in, 316–317
fields, 83–89
floating-point instructions, 206–212
floating-point instructions not in, D-22*f*
instruction classes, 157*f*
instruction encoding, 85*f*, 119*f*
instruction formats, 120, 146*f*
instruction set, 62, 159–160, 226, 236, D-9–D-16
machine language, 87–89
memory addresses, 70*f*
memory allocation for program and data, 106*f*
multiply in, 183
Pseudo, 226*f*

www.youseficlass.ir

register conventions, 107*f*
static multiple issue with, 324–328
Roofline model, 534–535, 536*f*, 537
with ceilings, 538*f*
computational roofline, 535, 537
illustrated, 534*f*
Opteron generations, 535
with overlapping areas shaded, 539*f*
peak floating-point performance, 538*f*
peak memory performance, 542*f*
with two kernels, 539*f*
Rotational delay, *See* Rotational latency
Rotational latency, 375
Rounding, 212–213
accurate, 212–213
bits, 214
with guard digits, 213*b*
IEEE 754 modes, 213–214
Row-major order, 211*b*–212*b*, 405
R-type, defined, 87*b*
R-type instructions, 248*b*–249*b*
datapath for, 256–259
datapath in operation for, 258*f*
RV32, 73*b*
RV64, 73*b*

S

Saturation, 177*b*

www.youseficlass.ir

sb (store byte), 64*f*
SB-type instruction format, 115
sc.d (store conditional), 64*f*
SCALAPAK, 223–224
Scaling
  strong, 497
  weak, 497
Scientific notation
  adding numbers in, 199
  defined, 191
  for reals, 191
sd (store doubleword), 64*f*
Search engines, 4
Secondary memory, 23
Sectors, 373–374
Seek, 374
Segmentation, 423*b*
Selector values, A-10
Semiconductors, 25–26
Send message routine, 521
Sensitivity list, A-23–A-24
Sequencers
  explicit, C-32
  implementing next-state function with, C-22–C-28
Sequential logic, A-4
Servers, 458.e6
  See also Desktop and server RISCs
  cost and capability, 5

www.youseficlass.ir

Service accomplishment, 410–411
Service interruption, 410
Set-associative caches, 395
See also Caches
address portions, 399*f*
block replacement strategies, 445
choice of, 444
four-way, 396*f*, 399
memory-block location, 395*f*
misses, 397*b*–398*b*
n-way, 395
two-way, 396*f*
Set less than instruction (slt), A-31
Setup time, A-52–A-53, A-53*f*
sh (store halfword), 64*f*
Shaders
defined, B-14
floating-point arithmetic, B-14
graphics, B-14–B-15
pixel example, B-15–B-17
Shading languages, B-14
Shadowing, 458.e4
Shared memory
See also Memory
as low-latency memory, B-21
caching in, B-58–B-60
CUDA, B-58
N-body and, B-66*f*

www.youseficlass.ir

per-CTA, B-39
SRAM banks, B-40
Shared memory multiprocessors (SMP), 509–514
defined, 492–493, 509–510
single physical address space, 509
synchronization, 510–513
Shift left logical immediate (slli), 90
Shift right arithmetic (srai), 90
Shift right logical immediate (srli), 90
Sign and magnitude, 192
Sign bit, 77
Sign extension, 246
defined, 78*b*
shortcut, 78–79
Signals
asserted, 242, A-4
control, 242, 255, 255, 255
deasserted, 242, A-4
Signed division, 187–188
Signed multiplication, 182
Signed numbers, 74–81
sign and magnitude, 75
treating as unsigned, 96
Significands, 193–194
addition, 198–199
multiplication, 201–205
Silicon, 25–26
as key hardware technology, 53

www.yousefclass.ir

crystal ingot, 26
defined, 25–26
wafers, 26
Silicon crystal ingot, 26
SIMD (Single Instruction Multiple Data), 498–499, 550–551
computers, 553.e1–553.e3
data vector, B-35
extensions, 553.e3
for loops and, 553.e2
massively parallel multiprocessors, 553.e1
small-scale, 553.e3
vector architecture, 500–502
in x86, 500
SIMMs (single inline memory modules), 473.e5*f*, 473.e4
Simple programmable logic devices (SPLDs), A-77
Simplicity, 65–67
Simultaneous multithreading (SMT), 507
support, 507*f*
thread-level parallelism, 507
unused issue slots, 507*f*
Single error correcting/Double error correcting (SEC/DEC), 412–416
Single instruction single data (SISD), 500, 504–506
Single precision
See also Double precision
binary representation, 196*b*
defined, 193
Single-clock-cycle pipeline diagrams, 287–290
illustrated, 289*f*

www.yousefclass.ir

Single-cycle datapaths
See also Datapaths
illustrated, 277*f*
instruction execution, 278*f*
Single-cycle implementation
control function for, 261
nonpipelined execution *versus* pipelined execution, 266*f*
non-use of, 261–262
penalty, 262
pipelined performance *versus*, 264*b*–265*b*
Single-instruction multiple-thread (SIMT), B-27–B-29
overhead, B-35
multithreaded warp scheduling, B-28*f*
processor architecture, B-28–B-29
warp execution and divergence, B-29–B-30
Single-program multiple data (SPMD), B-22
sll (shift left logical), 64*f*
slli (shift left logical immediate), 64*f*
Smalltalk-80, 162.e7, 162.e7
Smart phones, 7
Snooping protocol, 456–458
Snoopy cache coherence, 459.e16
Software optimization
via blocking, 405–409
Software
layers, 13*f*
multiprocessor, 492
parallel, 493

www.youseficlass.ir

as service, 7, 524, 550
systems, 13
Sort algorithms, 141*f*
Sort procedure, 135–140
See also Procedures
code for body, 136–138
full procedure, 139–140
passing parameters in, 138
preserving registers in, 138–139
procedure call, 138
register allocation for, 136
Sorting performance, B-54–B-55
Space allocation
on heap, 105–108
on stack, 104–105
SPARC
annulling branch, D-23–D-25
CASA, D-31–D-32
conditional branches, D-10–D-16
fast traps, D-30
floating-point operations, D-31
instructions, D-29–D-32
least significant bits, D-31*f*
multiple precision floating-point results, D-32
nonfaulting loads, D-32
overlapping integer operations, D-31
quadruple precision floating-point arithmetic, D-36
register windows, D-29–D-30

www.youseficlass.ir

support for LISP and Smalltalk, D-30
Sparse matrices, B-55–B-58
Sparse Matrix-Vector multiply (SpMV), B-55, B-57*f*, B-58
CUDA version, B-57*f*
serial code, B-57*f*
shared memory version, B-59*f*
Spatial locality, 366
large block exploitation of, 383
tendency, 369
SPEC, 54.e10–54.e11
CPU benchmark, 46–48
power benchmark, 48–49
SPEC89, 54.e10
SPEC92, 54.e11
SPEC95, 54.e11
SPEC2000, 54.e11
SPEC2006, 54.e11
SPECrate, 530
SPECratio, 47–48
Special function units (SFUs), B-35, B-50
defined, B-42–B-43
Speculation, 323–324
hardware-based, 331–333
implementation, 323
performance and, 323, 324
problems, 323
recovery mechanism, 323
Speed-up challenge

www.yousefclass.ir

balancing load, 497*b*–498*b*
bigger problem, 496*b*–497*b*
Spilling registers, 71*b*–72*b*, 99
Split algorithm, 544
Split caches, 389*b*
sra (shift right arithmetic), 64*f*
srai (shift right arithmetic immediate), 64*f*
srl (shift right logical), 64*f*
srli (shift right logical immediate), 64*f*
Stack architectures, 162.e3–162.e4
Stack pointers
adjustment, 102–104
defined, 99
values, 101*f*
Stacks
allocating space on, 104–105
for arguments, 99
defined, 99
pop, 99
push, 99, 102–104
Stalls, 270
avoiding with code reordering, 270*b*–271*b*
behavioral Verilog with detection, 345.e3–345.e8
data hazards and, 303–307
illustrations, 345.e20
insertion into pipeline, 305*f*
load-use, 308
memory, 391

www.youseficlass.ir

as solution to control hazard, 271
write-back scheme, 392
write buffer, 391
Standby spares, 458.e7
State
in 2-bit prediction scheme, 312
assignment, A-69, C-27
bits, C-8–C-10
exception, saving/restoring, 440
logic components, 241
specification of, 424*b*
State elements
clock and, 241
combinational logic and, 241
defined, 240–241, A-47
inputs, 241
register file, A-49*b*
in storing/accessing instructions, 244*f*
Static branch prediction, 324
Static data
segment, 105
Static multiple-issue processors, 322, 324–328
See also Multiple issue
control hazards and, 324–325
instruction sets, 324
with RISC-V ISA, 324–328
Static random access memories (SRAMs), 370, 371, A-57–A-66
array organization, A-61*f*

www.youseficlass.ir

basic structure, A-60*f*
defined, 19–22, A-57
fixed access time, A-57
large, A-58
read/write initiation, A-58
synchronous (SSRAMs), A-59
three-state buffers, A-58, A-59*f*
Static variables, 104*b*
Steady-state prediction, 312
Sticky bits, 214
Store buffers, 334*b*
Store byte, 109
Store-conditional doubleword, 122–123
Store doubleword, 70–71
Store instructions
See also Load instructions
access, B-41
base register, 254
compiling with, 71
conditional, 122–123
defined, 71*b*
EX stage, 284*f*
ID stage, 281*f*
IF stage, 281*f*
instruction dependency, 302*b*
MEM stage, 283*f*
unit for implementing, 247*f*
WB stage, 283*f*

www.youseficlass.ir

Store word, 113*b*
Stored program concept, 63
as computer principle, 88*b*
illustrated, 88*f*
principles, 159–160
Strcpy procedure, 110*b*–111*b*
See also Procedures
as leaf procedure, 111
pointers, 111
Stream benchmark, 540*b*
Streaming multiprocessor (SM), B-13
Streaming processors, B-34, B-49–B-50
array (SPA), B-41, B-46
Streaming SIMD Extension 2 (SSE2) floating-point architecture, 217
Streaming SIMD Extensions (SSE) and advanced vector extensions in x86, 217
Stretch computer, 347.e1*f*, 347.e1
Strings
defined, 109–111
in Java, 111–113
representation, 108*f*
Strip mining, 502*b*
Striping, 458.e3
Strong scaling, 497
Structural hazards, 267, 284
sub (subtract), 64*f*
Subnormals, 216
Subtraction, 174–177
See also Arithmetic

www.youseficlass.ir

binary, 174*b*–175*b*
floating-point, 206
negative number, 176
overflow, 176
Subword parallelism, 216–217, 344*f*, D-17
and matrix multiply, 218–222
Sum of products, A-11, A-12*b*
Supercomputers, 347.e2
defined, 5
SuperH, D-15, D-39–D-40
Superscalars
defined, 328, 347.e3–347.e4
dynamic pipeline scheduling, 328–329
multithreading options, 494
Supervisor Exception Cause Register (SCAUSE), 316
Supervisor exception program counter (SEPC), 316, 364, 439
address capture, 319–321
defined, 317–319
in restart determination, 316
Supervisor exception return (sret), 437
Supervisor Page Table Base Register (SPTBR), 427*f*
Supervisor Trap Vector (STVEC), 321*b*
Surfaces, B-41
sw (store word), 64*f*
Swap procedure, 134
See also Procedures
body code, 134–135
full, 135, 139–140

www.yousefclass.ir

register allocation, 134
Swap space, 426
Symbol tables, 126
Synchronization, 121–124, 544
barrier, B-18, B-20, B-34
defined, 510–513
lock, 121
overhead, reducing, 44–45
unlock, 121
Synchronizers
from D flip-flop, A-75*f*
defined, A-75
failure, A-75–A-76
Synchronous DRAM (SRAM), 371, A-59, A-64
Synchronous SRAM (SSRAM), A-59
Synchronous system, A-47–A-48
Syntax tree, 144.e2
System calls, defined, 364
Systems software, 13
SystemVerilog
cache controller, 459.e1–459.e4
cache data and tag modules, 459.e16
FSM, 459.e6*f*
simple cache block diagram, 459.e3*f*
type declarations, 459.e1*f*

T

Tablets, 7*f*

www.youseficlass.ir

Tags
defined, 376
in locating block, 399
page tables and, 426
size of, 401*b*–402*b*
Tail call, 107
Task identifiers, 438
Task parallelism, B-24
Task-level parallelism, 492
Tebibyte (TiB), 5
Telsa PTX ISA, B-31
arithmetic instructions, B-33
barrier synchronization, B-34
GPU thread instructions, B-32*f*
memory access instructions, 208
Temporal locality, 366
tendency, 369
Temporary registers, 68, 100
Terabyte (TB), 6*f*
defined, 5
Texture memory, B-40
Texture/processor cluster (TPC), B-47
TFLOPS multiprocessor, 553.e4–553.e5, 553.e5
Thrashing, 442
Thread blocks, 518*f*
creation, B-23
defined, B-19
managing, B-30

www.yousefclass.ir

memory sharing, B-20–B-21
synchronization, B-20–B-21
Thread parallelism, B-22
Threads
creation, B-23
CUDA, B-36
ISA, B-31–B-34
managing, B-30
memory latencies and, B-74*b*
multiple, per body, B-68–B-72
warps, B-27–B-28
Three Cs model, 447*b*
Three-state buffers, A-58, A-59*f*
Throughput
defined, 29–30
multiple issue and, 322
pipelining and, 264
Thumb, D-15*f*, D-38–D-39
Timing
asynchronous inputs, A-75–A-76
level-sensitive, A-74–A-75
methodologies, A-71–A-77
two-phase, A-74*f*
TLB misses, 431
See also Translation-lookaside buffer (TLB)
handling, 439–441
occurrence, 439
problem, 442

www.yousefclass.ir

Tomasulo's algorithm, 347.e2
Touchscreen, 19
Tournament branch predicators, 313–314
Tracks, 373–374
Transfer time, 375
Transistors, 25
Translation-lookaside buffer (TLB), 430–432, D-26–D-27, 473.e5
See also TLB misses
associativities, 432
illustrated, 431*f*
integration, 435
Intrinsity FastMATH, 432–435
typical values, 432
Transmit driver and NIC hardware time *versus* receive driver and NIC hardware time, 529.e7*f*
Tree-based parallel scan, B-62*f*
Truth tables, A-5
ALU control lines, C-5*f*
for control bits, 253
datapath control outputs, C-17*f*
datapath control signals, C-14*f*
defined, 253
example, A-5*b*
next-state output bits, C-15*f*
PLA implementation, A-13
Two's complement representation, 76
advantage, 77
negation shortcut, 78*b*–79*b*

www.youseficlass.ir

rule, 80*b*
sign extension shortcut, 79*b*–80*b*
Two-level logic, A-11–A-14
Two-phase clocking, A-74, A-74*f*
TX-2 computer, 553.e3

U

Unconditional branches, 93
Underflow, 192
Unicode
alphabets, 111
defined, 111
example alphabets, 112*f*
Unified GPU architecture, B-10–B-11
illustrated, B-11*f*
processor array, B-11–B-12
Uniform memory access (UMA), 510, B-9
multiprocessors, 510
Units
commit, 329, 334*b*
control, 239–240, 251–253, C-4–C-8, C-10*f*, C-12–C-13
defined, 213–214
floating point, 213–214
hazard detection, 303, 306–307
for load/store implementation, 247*f*
special function (SFUs), B-35, B-42–B-43, B-50
UNIVAC I, 54.e4*f*, 54.e3–54.e4
UNIX, 162.e7, 473.e10, 473.e13, 473.e13, 473.e14

www.yousefclass.ir

AT&T, 473.e14
Berkeley version (BSD), 473.e14
genius, 473.e16
history, 473.e13, 473.e14
Unlock synchronization, 121
Unsigned numbers, 74–81
Use latency
defined, 325–327
one-instruction, 325–327

V

Vacuum tubes, 25*f*
Valid bit, 376–378
Variables
C language, 104*b*
programming language, 67
register, 67
static, 104*b*
storage class, 104*b*
type, 104*b*
VAX architecture, 162.e3, 473.e6
Vector lanes, 502
Vector processors, 499–506
See also Processors
conventional code comparison, 501*b*–502*b*
instructions, 501
multimedia extensions and, 500–502
scalar *versus*, 502–503

www.youseficlass.ir

Vectored interrupts, 316
Verilog
behavioral definition of RISC-V ALU, A-25*f*
behavioral definition with bypassing, 345.e4*f*
behavioral definition with stalls for loads, 345.e6*f*
behavioral specification, A-21, 345.e1–345.e3
behavioral specification of multicycle MIPS design, 345.e12*f*
behavioral specification with simulation, 345.e1–345.e3
behavioral specification with stall detection, 345.e3–345.e8
behavioral specification with synthesis, 345.e8–345.e13
blocking assignment, A-24
branch hazard logic implementation, 345.e8
combinational logic, A-23–A-26
datatypes, A-21–A-23
defined, A-20–A-21
forwarding implementation, 345.e3
modules, A-23*f*
multicycle MIPS datapath, 345.e14*f*
nonblocking assignment, A-24
operators, A-22–A-23
program structure, A-23
reg, A-21, A-21
RISC-V ALU definition in, A-36–A-37
sensitivity list, A-23–A-24
sequential logic specification, A-55–A-57
structural specification, A-21
wire, A-21, A-21, A-22
Vertical microcode, C-32

www.youseficlass.ir

Very large-scale integrated (VLSI) circuits, 25
Very Long Instruction Word (VLIW)
defined, 324
first generation computers, 347.e4
processors, 324
VHDL, A-20–A-21
Video graphics array (VGA) controllers, B-3–B-4
Virtual addresses
causing page faults, 440
defined, 420–421
mapping from, 420–421
size, 422–423
Virtual machine monitors (VMMs)
defined, 416
implementing, 470*b*
laissez-faire attitude, 470
page tables, 441*b*
in performance improvement, 419
requirements, 418
Virtual machines (VMs), 416–419
benefits, 416–417
illusion, 441*b*
instruction set architecture support, 419
performance improvement, 419
for protection improvement, 416–417
Virtual memory, 419–443
See also Pages
address translation, 420–421, 430–432

www.youseficlass.ir

integration, 435–437
for large virtual addresses, 428–429
mechanism, 442
motivations, 419–420
page faults, 420–421, 426
protection implementation, 437–439
segmentation, 423*b*
summary, 441–443
virtualization of, 441*b*
writes, 430
Virtualizable hardware, 418
Virtually addressed caches, 436
Visual computing, B-3
Volatile memory, 22

W

Wafers, 26
defects, 26–27
dies, 27, 27, 27, 28
yield, 27
Warehouse Scale Computers (WSCs), 7, 521–526, 550
Warps, B-27–B-28
Weak scaling, 497
Wear levelling, 373
While loops, 94*b*–95*b*
Whirlwind, 473.e1
Wide area networks (WANs), 24
See also Networks

www.youseficlass.ir

Wide immediate operands, 113–114
Words
accessing, 68
defined, 67
double, 151
load, 69, 71
quad, 151
store, 71*b*
Working set, 442
World Wide Web, 4
Worst-case delay, 262
Write buffers
defined, 387
stalls, 383
write-back cache, 387
Write invalidate protocols, 456
Write serialization, 455–456
Write-back caches
See also Caches
advantages, 446
cache coherency protocol, 459.e4
complexity, 387
defined, 386, 446
stalls, 391
write buffers, 387
Write-back stage
control line, 292*f*
load instruction, 282

www.yousefclass.ir

store instruction, 284
Writes
complications, 386*b*–387*b*
expense, 442
handling, 385–387
memory hierarchy handling of, 333–334
schemes, 386
virtual memory, 429
write-back cache, 386, 387
write-through cache, 386, 387
Write-stall cycles, 391
Write-through caches
See also Caches
advantages, 446
defined, 385, 446
tag mismatch, 386

X

x86, 146–155
Advanced Vector Extensions in, 217–218
brief history, 162.e5–162.e6
conclusion, 154–155
data addressing modes, 149–151
evolution, 96
first address specifier encoding, 155*f*
instruction encoding, 153–154
instruction formats, 154*f*
instruction set growth, 162*f*

www.yousefclass.ir

instruction types, 152*f*
integer operations, 151–152
registers, 149–151
SIMD in, 498–499
Streaming SIMD Extensions in, 217–218
typical instructions/functions, 154*f*
typical operations, 153*f*
unique, D-36–D-38
Xerox Alto computer, 54.e7–54.e9
XMM, 217
xor (exclusive or), 64*f*
xori (exclusive or immediate), 64*f*

Y

Yahoo! Cloud Serving Benchmark (YCSB), 532
Yield, 27
YMM, 218

Z

Zettabyte, 6*f*

www.youseficlass.ir

RISC-V Reference Data Card (“Green Card”)

www.youseficlass.ir

RISC-V Reference Data Card ("Green Card") 1. Pull along perforation to separate card 2. Fold bottom side (columns 3 and 4) together

RISC-V Reference Data ①

RV64I BASE INTEGER INSTRUCTIONS, in alphabetical order

MNEMONIC	FMT	NAME	DESCRIPTION (in Verilog)	NOTE
add,addw	R	ADD (Word)	R[rd] = R[rs1] + R[rs2]	1)
addi,addiw	I	ADD Immediate (Word)	R[rd] = R[rs1] + imm	1)
and	R	AND	R[rd] = R[rs1] & R[rs2]	
andi	I	AND Immediate	R[rd] = R[rs1] & imm	
auipc	U	Add Upper Immediate to PC	R[rd] = PC + {imm, 12'b0}	
beq	SB	Branch EQual	if(R[rs1]==R[rs2) PC=PC+{imm,1b'0}	
bge	SB	Branch Greater than or Equal	if(R[rs1]>=R[rs2) PC=PC+{imm,1b'0}	
bgeu	SB	Branch ≥ Unsigned	if(R[rs1]>=R[rs2) PC=PC+{imm,1b'0}	2)
blt	SB	Branch Less Than	if(R[rs1]<R[rs2) PC=PC+{imm,1b'0}	
bltu	SB	Branch Less Than Unsigned	if(R[rs1]<R[rs2) PC=PC+{imm,1b'0}	2)
bne	SB	Branch Not Equal	if(R[rs1]!=R[rs2) PC=PC+{imm,1b'0}	
csrrc	I	Cont./Stat.RegRead&Clear	R[rd] = CSR;CSR = CSR & ~R[rs1]	
csrrci	I	Cont./Stat.RegRead&Clear Imm	R[rd] = CSR;CSR = CSR & ~imm	
csrrs	I	Cont./Stat.RegRead&Set	R[rd] = CSR; CSR = CSR | R[rs1]	
csrrsi	I	Cont./Stat.RegRead&Set Imm	R[rd] = CSR; CSR = CSR | imm	
csrrw	I	Cont./Stat.RegRead&Write	R[rd] = CSR; CSR = R[rs1]	
csrrwi	I	Cont./Stat.Reg Read&Write Imm	R[rd] = CSR; CSR = imm	
ebreak	I	Environment BREAK	Transfer control to debugger	
ecall	I	Environment CALL	Transfer control to operating system	
fence	I	Synch thread	Synchronizes threads	
fence.i	I	Synch Instr & Data	Synchronizes writes to instruction stream	
jal	UJ	Jump & Link	R[rd] = PC+4; PC = PC + {imm,1b'0}	
jalr	I	Jump & Link Register	R[rd] = PC+4; PC = R[rs1]+imm	3)
lb	I	Load Byte	R[rd] = {56'bM[](7),M[R[rs1]+imm](7:0)}	4)
lbu	I	Load Byte Unsigned	R[rd] = {56'b0,M[R[rs1]+imm](7:0)}	
ld	I	Load Doubleword	R[rd] = M[R[rs1]+imm](63:0)	
lh	I	Load Halfword	R[rd] = {48'bM[](15),M[R[rs1]+imm](15:0)}	4)
lhu	I	Load Halfword Unsigned	R[rd] = {48'b0,M[R[rs1]+imm](15:0)}	
lui	U	Load Upper Immediate	R[rd] = {32b'imm<31>, imm, 12'b0}	
lw	I	Load Word	R[rd] = {32'bM[](31),M[R[rs1]+imm](31:0)}	4)
lwu	I	Load Word Unsigned	R[rd] = {32'b0,M[R[rs1]+imm](31:0)}	
or	R	OR	R[rd] = R[rs1] | R[rs2]	
ori	I	OR Immediate	R[rd] = R[rs1] | imm	
sb	S	Store Byte	M[R[rs1]+imm](7:0) = R[rs2](7:0)	
sd	S	Store Doubleword	M[R[rs1]+imm](63:0) = R[rs2](63:0)	
sh	S	Store Halfword	M[R[rs1]+imm](15:0) = R[rs2](15:0)	
sll,sllw	R	Shift Left (Word)	R[rd] = R[rs1] << R[rs2]	1)
slli,slliw	I	Shift Left Immediate (Word)	R[rd] = R[rs1] << imm	1)
slt	R	Set Less Than	R[rd] = (R[rs1] < R[rs2]) ? 1 : 0	
slti	I	Set Less Than Immediate	R[rd] = (R[rs1] < imm) ? 1 : 0	
sltiu	I	Set < Immediate Unsigned	R[rd] = (R[rs1] < imm) ? 1 : 0	2)
sltu	R	Set Less Than Unsigned	R[rd] = (R[rs1] < R[rs2]) ? 1 : 0	2)
sra,sraw	R	Shift Right Arithmetic (Word)	R[rd] = R[rs1] >> R[rs2]	1,5)
srai,sraiw	I	Shift Right Arith Imm (Word)	R[rd] = R[rs1] >> imm	1,5)
srl,srlw	R	Shift Right (Word)	R[rd] = R[rs1] >> R[rs2]	1)
srli,srliw	I	Shift Right Immediate (Word)	R[rd] = R[rs1] >> imm	1)
sub,subw	R	SUBtract (Word)	R[rd] = R[rs1] – R[rs2]	1)
sw	S	Store Word	M[R[rs1]+imm](31:0) = R[rs2](31:0)	
xor	R	XOR	R[rd] = R[rs1] ^ R[rs2]	
xori	I	XOR Immediate	R[rd] = R[rs1] ^ imm	

Notes: 1) The Word version only operates on the rightmost 32 bits of a 64-bit registers
2) Operation assumes unsigned integers (instead of 2's complement)
3) The least significant bit of the branch address in jalr is set to 0
4) (signed) Load instructions extend the sign bit of data to fill the 64-bit register
5) Replicates the sign bit to fill in the leftmost bits of the result during right shift
6) Multiply with one operand signed and one unsigned
7) The Single version does a single-precision operation using the rightmost 32 bits of a 64-bit F register
8) Classify writes a 10-bit mask to show which properties are true (e.g., -inf, -0, +0, +inf, denorm, ...)
9) Atomic memory operation; nothing else can interpose itself between the read and the write of the memory location
The immediate field is sign-extended in RISC-V

ARITHMETIC CORE INSTRUCTION SET ②

RV64M Multiply Extension

MNEMONIC	FMT	NAME	DESCRIPTION (in Verilog)	NOTE
mul,mulw	R	MULtiply (Word)	R[rd] = (R[rs1] * R[rs2])(63:0)	1)
mulh	R	MULtiply High	R[rd] = (R[rs1] * R[rs2])(127:64)	
mulhu	R	MULtiply High Unsigned	R[rd] = (R[rs1] * R[rs2])(127:64)	2)
mulhsu	R	MULtiply upper Half Sign/Uns	R[rd] = (R[rs1] * R[rs2])(127:64)	6)
div,divw	R	DIVide (Word)	R[rd] = (R[rs1] / R[rs2])	1)
divu	R	DIVide Unsigned	R[rd] = (R[rs1] / R[rs2])	2)
rem,remw	R	REMainder (Word)	R[rd] = (R[rs1] % R[rs2])	1)
remu,remuw	R	REMainder Unsigned (Word)	R[rd] = (R[rs1] % R[rs2])	1,2)

RV64F and RV64D Floating-Point Extensions

fld,flw	I	Load (Word)	F[rd] = M[R[rs1]+imm]	1)
fsd,fsw	S	Store (Word)	M[R[rs1]+imm] = F[rd]	1)
fadd.s,fadd.d	R	ADD	F[rd] = F[rs1] + F[rs2]	7)
fsub.s,fsub.d	R	SUBtract	F[rd] = F[rs1] – F[rs2]	7)
fmul.s,fmul.d	R	MULtiply	F[rd] = F[rs1] * F[rs2]	7)
fdiv.s,fdiv.d	R	DIVide	F[rd] = F[rs1] / F[rs2]	7)
fsqrt.s,fsqrt.d	R	SQuare RooT	F[rd] = sqrt(F[rs1])	7)
fmadd.s,fmadd.d	R	Multiply-ADD	F[rd] = F[rs1] * F[rs2] + F[rs3]	7)
fmsub.s,fmsub.d	R	Multiply-SUBtract	F[rd] = F[rs1] * F[rs2] - F[rs3]	7)
fnmadd.s,fnmadd.d	R	Negative Multiply-ADD	F[rd] = –(F[rs1] * F[rs2] + F[rs3])	7)
fnmsub.s,fnmsub.d	R	Negative Multiply-SUBtract	F[rd] = –(F[rs1] * F[rs2] – F[rs3])	7)
fsgnj.s,fsgnj.d	R	SiGN source	F[rd] = { F[rs2]<63>,F[rs1]<62:0>}	7)
fsgnjn.s,fsgnjn.d	R	Negative SiGN source	F[rd] = { (~F[rs2]<63>), F[rs1]<62:0>}	7)
fsgnjx.s,fsgnjx.d	R	Xor SiGN source	F[rd] = {F[rs2]<63>^F[rs1]<63>, F[rs1]<62:0>}	7)
fmin.s,fmin.d	R	MINimum	F[rd] = (F[rs1] < F[rs2]) ? F[rs1] : F[rs2]	7)
fmax.s,fmax.d	R	MAXimum	F[rd] = (F[rs1] > F[rs2]) ? F[rs1] : F[rs2]	7)
feq.s,feq.d	R	Compare Float EQual	R[rd] = (F[rs1]== F[rs2]) ? 1 : 0	7)
flt.s,flt.d	R	Compare Float Less Than	R[rd] = (F[rs1]< F[rs2]) ? 1 : 0	7)
fle.s,fle.d	R	Compare Float Less than or =	R[rd] = (F[rs1]<= F[rs2]) ? 1 : 0	7)
fclass.s,fclass.d	R	Classify Type	R[rd] = class(F[rs1])	7,8)
fmv.s.x,fmv.d.x	R	Move from Integer	F[rd] = R[rs1]	7)
fmv.x.s,fmv.x.d	R	Move to Integer	R[rd] = F[rs1]	7)
fcvt.s.d	R	Convert to SP from DP	F[rd] = single(F[rs1])	
fcvt.d.s	R	Convert to DP from SP	F[rd] = double(F[rs1])	
fcvt.s.w,fcvt.d.w	R	Convert from 32b Integer	F[rd] = float(R[rs1](31:0))	7)
fcvt.s.l,fcvt.d.l	R	Convert from 64b Integer	F[rd] = float(R[rs1](63:0))	7)
fcvt.s.wu,fcvt.d.wu	R	Convert from 32b Int Unsigned	F[rd] = float(R[rs1](31:0))	2,7)
fcvt.s.lu,fcvt.d.lu	R	Convert from 64b Int Unsigned	F[rd] = float(R[rs1](63:0))	2,7)
fcvt.w.s,fcvt.w.d	R	Convert to 32b Integer	R[rd](31:0) = integer(F[rs1])	7)
fcvt.l.s,fcvt.l.d	R	Convert to 64b Integer	R[rd](63:0) = integer(F[rs1])	7)
fcvt.wu.s,fcvt.wu.d	R	Convert to 32b Int Unsigned	R[rd](31:0) = integer(F[rs1])	2,7)
fcvt.lu.s,fcvt.lu.d	R	Convert to 64b Int Unsigned	R[rd](63:0) = integer(F[rs1])	2,7)

RV64A Atomic Extension

amoadd.w,amoadd.d	R	ADD	R[rd] = M[R[rs1]], M[R[rs1]] = M[R[rs1]] + R[rs2]	9)
amoand.w,amoand.d	R	AND	R[rd] = M[R[rs1]], M[R[rs1]] = M[R[rs1]] & R[rs2]	9)
amomax.w,amomax.d	R	MAXimum	R[rd] = M[R[rs1]], if (R[rs2] > M[R[rs1]]) M[R[rs1]] = R[rs2]	9)
amomaxu.w,amomaxu.d	R	MAXimum Unsigned	R[rd] = M[R[rs1]], if (R[rs2] > M[R[rs1]]) M[R[rs1]] = R[rs2]	2,9)
amomin.w,amomin.d	R	MINimum	R[rd] = M[R[rs1]], if (R[rs2] < M[R[rs1]]) M[R[rs1]] = R[rs2]	9)
amominu.w,amominu.d	R	MINimum Unsigned	R[rd] = M[R[rs1]], if (R[rs2] < M[R[rs1]]) M[R[rs1]] = R[rs2]	2,9)
amoor.w,amoor.d	R	OR	R[rd] = M[R[rs1]], M[R[rs1]] = M[R[rs1]] | R[rs2]	9)
amoswap.w,amoswap.d	R	SWAP	R[rd] = M[R[rs1]], M[R[rs1]] = R[rs2]	9)
amoxor.w,amoxor.d	R	XOR	R[rd] = M[R[rs1]], M[R[rs1]] = M[R[rs1]] ^ R[rs2]	9)
lr.w,lr.d	R	Load Reserved	R[rd] = M[R[rs1]], reservation on M[R[rs1]]	
sc.w,sc.d	R	Store Conditional	if reserved, M[R[rs1]] = R[rs2], R[rd] = 0; else R[rd] = 1	

CORE INSTRUCTION FORMATS

	31 27	26 25	24 20	19 15	14 12	11 7	6 0
R	funct7		rs2	rs1	funct3	rd	Opcode
I	imm[11:0]			rs1	funct3	rd	Opcode
S	imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode
SB	imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]					rd	opcode
UJ	imm[20|10:1|11|19:12]					rd	opcode

© 2018 by Elsevier, Inc. All rights reserved. From Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface: RISC-V Edition*

www.youseficlass.ir

PSEUDO INSTRUCTIONS ③

MNEMONIC	NAME	DESCRIPTION	USES
beqz	Branch = zero	if(R[rs1]==0) PC=PC+{imm,1b'0}	beq
bnez	Branch ≠ zero	if(R[rs1]!=0) PC=PC+{imm,1b'0}	bne
fabs.s,fabs.d	Absolute Value	F[rd] = (F[rs1]< 0) ? –F[rs1] : F[rs1]	fsgnx
fmv.s,fmv.d	FP Move	F[rd] = F[rs1]	fsgnj
fneg.s,fneg.d	FP negate	F[rd] = –F[rs1]	fsgnjn
j	Jump	PC = {imm,1b'0}	jal
jr	Jump register	PC = R[rs1]	jalr
la	Load address	R[rd] = address	auipc
li	Load imm	R[rd] = imm	addi
mv	Move	R[rd] = R[rs1]	addi
neg	Negate	R[rd] = –R[rs1]	sub
nop	No operation	R[0] = R[0]	addi
not	Not	R[rd] = ~R[rs1]	xori
ret	Return	PC = R[1]	jalr
seqz	Set = zero	R[rd] = (R[rs1]== 0) ? 1 : 0	sltiu
snez	Set ≠ zero	R[rd] = (R[rs1]!= 0) ? 1 : 0	sltu

OPCODES IN NUMERICAL ORDER BY OPCODE

MNEMONIC	FMT	OPCODE	FUNCT3	FUNCT7 OR IMM	HEXADECIMAL
lb	I	0000011	000		03/0
lh	I	0000011	001		03/1
lw	I	0000011	010		03/2
ld	I	0000011	011		03/3
lbu	I	0000011	100		03/4
lhu	I	0000011	101		03/5
lwu	I	0000011	110		03/6
fence	I	0001111	000		0F/0
fence.i	I	0001111	001		0F/1
addi	I	0010011	000		13/0
slli	I	0010011	001	0000000	13/1/00
slti	I	0010011	010		13/2
sltiu	I	0010011	011		13/3
xori	I	0010011	100		13/4
srli	I	0010011	101	0000000	13/5/00
srai	I	0010011	101	0100000	13/5/20
ori	I	0010011	110		13/6
andi	I	0010011	111		13/7
auipc	U	0010111			17
addiw	I	0011011	000		1B/0
slliw	I	0011011	001	0000000	1B/1/00
srliw	I	0011011	101	0000000	1B/5/00
sraiw	I	0011011	101	0100000	1B/5/20
sb	S	0100011	000		23/0
sh	S	0100011	001		23/1
sw	S	0100011	010		23/2
sd	S	0100011	011		23/3
add	R	0110011	000	0000000	33/0/00
sub	R	0110011	000	0100000	33/0/20
sll	R	0110011	001	0000000	33/1/00
slt	R	0110011	010	0000000	33/2/00
sltu	R	0110011	011	0000000	33/3/00
xor	R	0110011	100	0000000	33/4/00
srl	R	0110011	101	0000000	33/5/00
sra	R	0110011	101	0100000	33/5/20
or	R	0110011	110	0000000	33/6/00
and	R	0110011	111	0000000	33/7/00
lui	U	0110111			37
addw	R	0111011	000	0000000	3B/0/00
subw	R	0111011	000	0100000	3B/0/20
sllw	R	0111011	001	0000000	3B/1/00
srlw	R	0111011	101	0000000	3B/5/00
sraw	R	0111011	101	0100000	3B/5/20
beq	SB	1100011	000		63/0
bne	SB	1100011	001		63/1
blt	SB	1100011	100		63/4
bge	SB	1100011	101		63/5
bltu	SB	1100011	110		63/6
bgeu	SB	1100011	111		63/7
jalr	I	1100111	000		67/0
jal	UJ	1101111			6F
ecall	I	1110011	000	000000000000	73/0/000
ebreak	I	1110011	000	000000000001	73/0/001
CSRRW	I	1110011	001		73/1
CSRRS	I	1110011	010		73/2
CSRRC	I	1110011	011		73/3
CSRRWI	I	1110011	101		73/5
CSRRSI	I	1110011	110		73/6
CSRRCI	I	1110011	111		73/7

REGISTER NAME, USE, CALLING CONVENTION ④

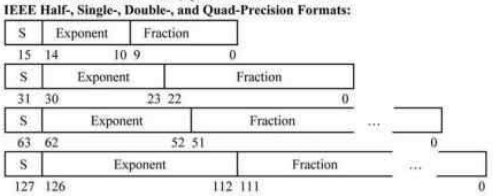
REGISTER	NAME	USE	SAVER
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	--
x4	tp	Thread pointer	--
x5–x7	t0–t2	Temporaries	Caller
x8	s0/fp	Saved register/Frame pointer	Callee
x9	s1	Saved register	Callee
x10–x11	a0–a1	Function arguments/Return values	Caller
x12–x17	a2–a7	Function arguments	Caller
x18–x27	s2–s11	Saved registers	Callee
x28–x31	t3–t6	Temporaries	Caller
f0–f7	ft0–ft7	FP Temporaries	Caller
f8–f9	fs0–fs1	FP Saved registers	Callee
f10–f11	fa0–fa1	FP Function arguments/Return values	Caller
f12–f17	fa2–fa7	FP Function arguments	Caller
f18–f27	fs2–fs11	FP Saved registers	Callee
f28–f31	ft8–ft11	R[rd] = R[rs1] + R[rs2]	Caller

IEEE 754 FLOATING-POINT STANDARD

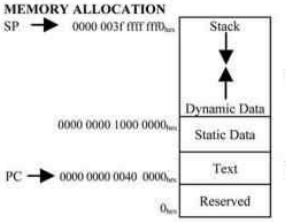
$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$

where Half-Precision Bias = 15, Single-Precision Bias = 127,
Double-Precision Bias = 1023, Quad-Precision Bias = 16383

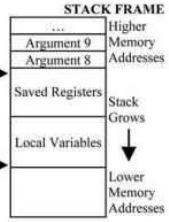
IEEE Half-, Single-, Double-, and Quad-Precision Formats:



MEMORY ALLOCATION



STACK FRAME



SIZE PREFIXES AND SYMBOLS

SIZE	PREFIX	SYMBOL	SIZE	PREFIX	SYMBOL
10^3	Kilo-	K	2^{10}	Kibi-	Ki
10^6	Mega-	M	2^{20}	Mebi-	Mi
10^9	Giga-	G	2^{30}	Gibi-	Gi
10^{12}	Tera-	T	2^{40}	Tebi-	Ti
10^{15}	Peta-	P	2^{50}	Pebi-	Pi
10^{18}	Exa-	E	2^{60}	Exbi-	Ei
10^{21}	Zetta-	Z	2^{70}	Zebi-	Zi
10^{24}	Yotta-	Y	2^{80}	Yobi-	Yi
10^{-3}	milli-	m	10^{-15}	femto-	f
10^{-6}	micro-	μ	10^{-18}	atto-	a
10^{-9}	nano-	n	10^{-21}	zepto-	z
10^{-12}	pico-	p	10^{-24}	yocto-	y

RISC-V Reference Data Card ("Green Card") 1. Pull along perforation to separate card 2. Fold bottom side (columns 3 and 4) together

© 2018 by Elsevier, Inc. All rights reserved. From Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface: RISC-V Edition*

www.yousefclass.ir

目录

Title page 2
Table of Contents 6
In Praise of Computer Organization and Design: The Hardware/Software Interface 16
Copyright 19
Dedication 22
Acknowledgments 23
Preface 24
About This Book 24
About the Other Book 25
Why RISC-V for This Edition? 26
Changes for the Fifth Edition 28
Instructor Support 32
Concluding Remarks 32
Acknowledgments 33
1. Computer Abstractions and Technology 39
Abstract 39
1.1 Introduction 40
1.2 Eight Great Ideas in Computer Architecture 52
1.3 Below Your Program 58
1.4 Under the Covers 67
1.5 Technologies for Building Processors and Memory 84
1.6 Performance 91
1.7 The Power Wall 107
1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors 111
1.9 Real Stuff: Benchmarking the Intel Core i7 120
1.10 Fallacies and Pitfalls 124
1.11 Concluding Remarks 129
Historical Perspective and Further Reading 134
1.12 Historical Perspective and Further Reading 134

www.youseficlass.ir

1.13 Exercises 155
2. Instructions: Language of the Computer 163
Abstract 163
2.1 Introduction 166
2.2 Operations of the Computer Hardware 169
2.3 Operands of the Computer Hardware 172
2.4 Signed and Unsigned Numbers 184
2.5 Representing Instructions in the Computer 195
2.6 Logical Operations 206
2.7 Instructions for Making Decisions 210
2.8 Supporting Procedures in Computer Hardware 218
2.9 Communicating with People 233
2.10 RISC-V Addressing for Wide Immediates and Addresses 239
2.11 Parallelism and Instructions: Synchronization 249
2.12 Translating and Starting a Program 254
2.13 A C Sort Example to Put it All Together 270
2.14 Arrays versus Pointers 279
Advanced Material: Compiling C and Interpreting Java 284
2.15 Advanced Material: Compiling C and Interpreting Java 284
2.16 Real Stuff: MIPS Instructions 320
2.17 Real Stuff: x86 Instructions 323
2.18 Real Stuff: The Rest of the RISC-V Instruction Set 336
2.19 Fallacies and Pitfalls 338
2.20 Concluding Remarks 341
Historical Perspective and Further Reading 345
2.22 Historical Perspective and Further Reading 346
2.22 Exercises 358
3. Arithmetic for Computers 368
Abstract 368
3.1 Introduction 370
3.2 Addition and Subtraction 371
3.3 Multiplication 375
3.4 Division 385
3.5 Floating Point 398
3.6 Parallelism and Computer Arithmetic: Subword Parallelism 434

www.youseficlass.ir

3.7 Real Stuff: Streaming SIMD Extensions and Advanced Vector Extensions in x86 435
3.8 Going Faster: Subword Parallelism and Matrix Multiply 437
3.9 Fallacies and Pitfalls 442
3.10 Concluding Remarks 447
Historical Perspective and Further Reading 451
Historical Perspective and Further Reading 451
3.12 Exercises 465
4. The Processor 472
Abstract 472
4.1 Introduction 474
4.2 Logic Design Conventions 481
4.3 Building a Datapath 486
4.4 A Simple Implementation Scheme 500
4.5 An Overview of Pipelining 518
4.6 Pipelined Datapath and Control 543
4.7 Data Hazards: Forwarding versus Stalling 569
4.8 Control Hazards 588
4.9 Exceptions 602
4.10 Parallelism via Instructions 612
4.11 Real Stuff: The ARM Cortex-A53 and Intel Core i7 Pipelines 637
4.12 Going Faster: Instruction-Level Parallelism and Matrix Multiply 650
Advanced Topic: An Introduction to Digital Design Using a Hardware Design Language to Describe and Model a Pipeline and More Pipelining Illustrations 655
4.13 Advanced Topic: An Introduction to Digital Design Using a Hardware Design Language to Describe and Model a Pipeline and More Pipelining Illustrations 656
4.14 Fallacies and Pitfalls 702
4.15 Concluding Remarks 704
Historical Perspective and Further Reading 707
4.16 Historical Perspective and Further Reading 707
4.17 Exercises 715

www.youseficlass.ir

5. Large and Fast: Exploiting Memory Hierarchy 734
Abstract 734
5.1 Introduction 737
5.2 Memory Technologies 745
5.3 The Basics of Caches 754
5.4 Measuring and Improving Cache Performance 776
5.5 Dependable Memory Hierarchy 804
5.6 Virtual Machines 813
5.7 Virtual Memory 818
5.8 A Common Framework for Memory Hierarchy 857
5.9 Using a Finite-State Machine to Control a Simple Cache 867
5.10 Parallelism and Memory Hierarchy: Cache Coherence 875
Parallelism and Memory Hierarchy: Redundant Arrays of Inexpensive Disks 880
5.11 Parallelism and Memory Hierarchy: Redundant Arrays of Inexpensive Disks 881
Advanced Material: Implementing Cache Controllers 892
5.12 Advanced Material: Implementing Cache Controllers 892
5.13 Real Stuff: The ARM Cortex-A53 and Intel Core i7 Memory Hierarchies 912
5.14 Real Stuff: The Rest of the RISC-V System and Special Instructions 921
5.15 Going Faster: Cache Blocking and Matrix Multiply 922
5.16 Fallacies and Pitfalls 925
5.17 Concluding Remarks 932
Historical Perspective and Further Reading 934
5.18 Historical Perspective and Further Reading 935
5.19 Exercises 959
6. Parallel Processors from Client to Cloud 979
Abstract 979
6.1 Introduction 981
6.2 The Difficulty of Creating Parallel Processing Programs 986
6.3 SISD, MIMD, SIMD, SPMD, and Vector 993
6.4 Hardware Multithreading 1007
6.5 Multicore and Other Shared Memory Multiprocessors 1016

www.youseficlass.ir

6.6 Introduction to Graphics Processing Units 1022
6.7 Clusters, Warehouse Scale Computers, and Other Message-Passing Multiprocessors 1033
6.8 Introduction to Multiprocessor Network Topologies 1043
Communicating to the Outside World: Cluster Networking 1049
6.9 Communicating to the Outside World: Cluster Networking 1050
6.10 Multiprocessor Benchmarks and Performance Models 1064
6.11 Real Stuff: Benchmarking and Rooflines of the Intel Core i7 960 and the NVIDIA Tesla GPU 1082
6.12 Going Faster: Multiple Processors and Matrix Multiply 1089
6.13 Fallacies and Pitfalls 1095
6.14 Concluding Remarks 1098
Historical Perspective and Further Reading 1102
6.15 Historical Perspective and Further Reading 1103
6.16 Exercises 1119
Appendix 1135
Appendix A. The Basics of Logic Design 1136
A.1 Introduction 1136
A.2 Gates, Truth Tables, and Logic Equations 1138
A.3 Combinational Logic 1144
A.4 Using a Hardware Description Language 1166
A.5 Constructing a Basic Arithmetic Logic Unit 1175
A.6 Faster Addition: Carry Lookahead 1196
A.7 Clocks 1210
A.8 Memory Elements: Flip-Flops, Latches, and Registers 1213
A.9 Memory Elements: SRAMs and DRAMs 1227
A.10 Finite-State Machines 1240
A.11 Timing Methodologies 1249
A.12 Field Programmable Devices 1256
A.13 Concluding Remarks 1259
A.14 Exercises 1260
Appendix B. Graphics and Computing GPUs 1271
B.1 Introduction 1271
B.2 GPU System Architectures 1277
B.3 Programming GPUs 1287

www.youseficlass.ir

B.4 Multithreaded Multiprocessor Architecture 1308
B.5 Parallel Memory System 1326
B.6 Floating-point Arithmetic 1332
B.7 Real Stuff: The NVIDIA GeForce 8800 1340
B.8 Real Stuff: Mapping Applications to GPUs 1352
B.9 Fallacies and Pitfalls 1376
B.10 Concluding Remarks 1381
B.11 Historical Perspective and Further Reading 1382
Further Reading 1389
Appendix C. Mapping Control to Hardware 1393
C.1 Introduction 1393
C.2 Implementing Combinational Control Units 1394
C.3 Implementing Finite-State Machine Control 1400
C.4 Implementing the Next-State Function with a Sequencer 1417
C.5 Translating a Microprogram to Hardware 1426
C.6 Concluding Remarks 1431
C.7 Exercises 1432
Appendix D. A Survey of RISC Architectures for Desktop, Server, and Embedded Computers 1434
D.1 Introduction 1435
D.2 Addressing Modes and Instruction Formats 1437
D.3 Instructions: The MIPS Core Subset 1444
D.4 Instructions: Multimedia Extensions of the Desktop/Server RISCs 1453
D.5 Instructions: Digital Signal-Processing Extensions of the Embedded RISCs 1455
D.6 Instructions: Common Extensions to MIPS Core 1456
D.7 Instructions Unique to MIPS-64 1462
D.8 Instructions Unique to Alpha 1466
D.9 Instructions Unique to SPARC v9 1468
D.10 Instructions Unique to PowerPC 1472
D.11 Instructions Unique to PA-RISC 2.0 1473
D.12 Instructions Unique to ARM 1477
D.13 Instructions Unique to Thumb 1478
D.14 Instructions Unique to SuperH 1480

www.youseficlass.ir

D.15 Instructions Unique to M32R 1481
D.16 Instructions Unique to MIPS-16 1482
D.17 Concluding Remarks 1484
Further Reading 1488
Answers to Check Yourself 1490
Chapter 1 1490
Chapter 2 1490
Chapter 3 1491
Chapter 4 1491
Chapter 5 1492
Chapter 6 1492
Glossary 1494
Further Reading 1524
Index 1541
RISC-V Reference Data Card (“Green Card”) 1656

www.youseficlass.ir